

BikeNetKit: Developing an open-source bicycle network software suite for urban planners

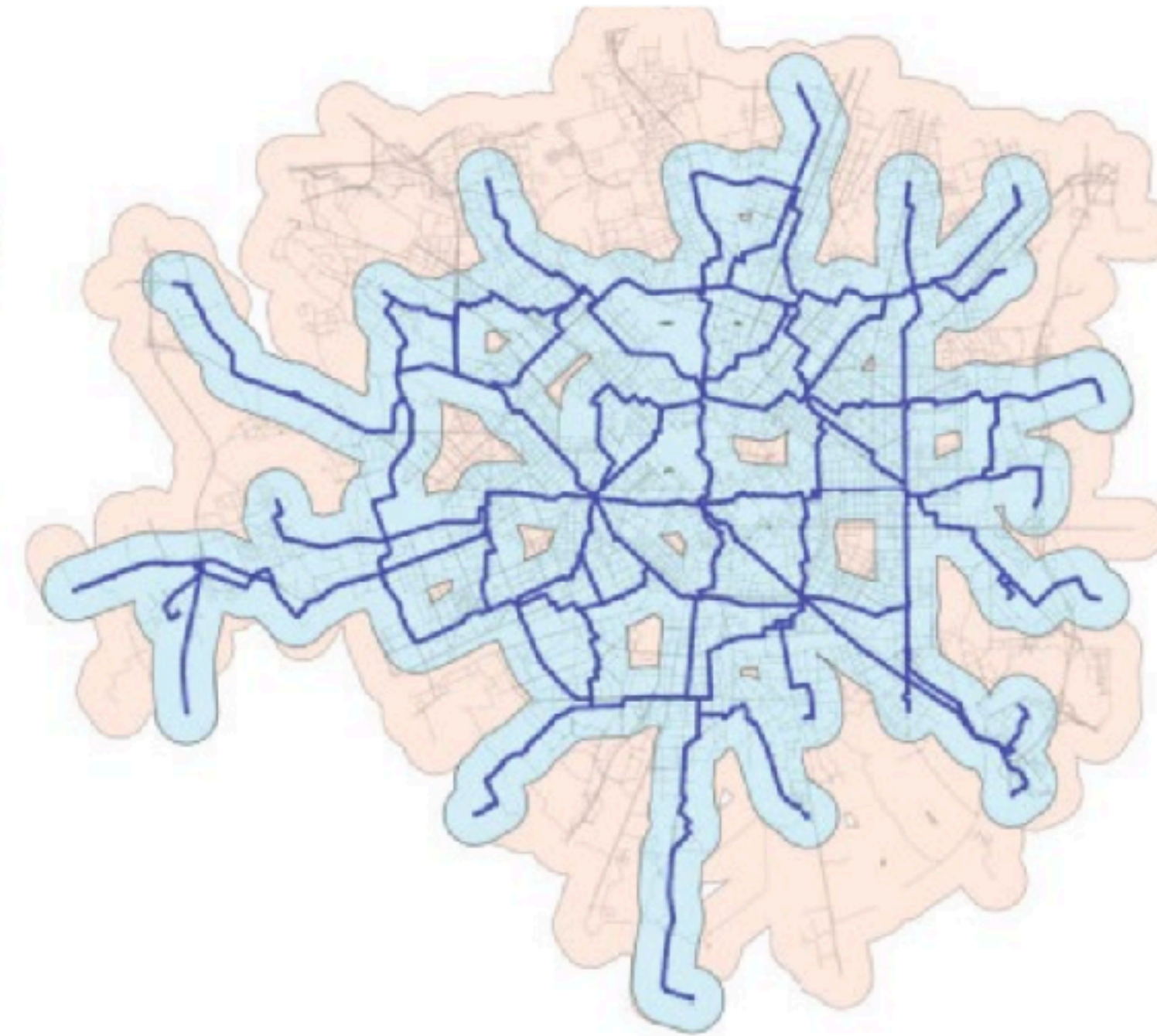
Manuel Knepper

Anastassia Vybornova

Michael Szell



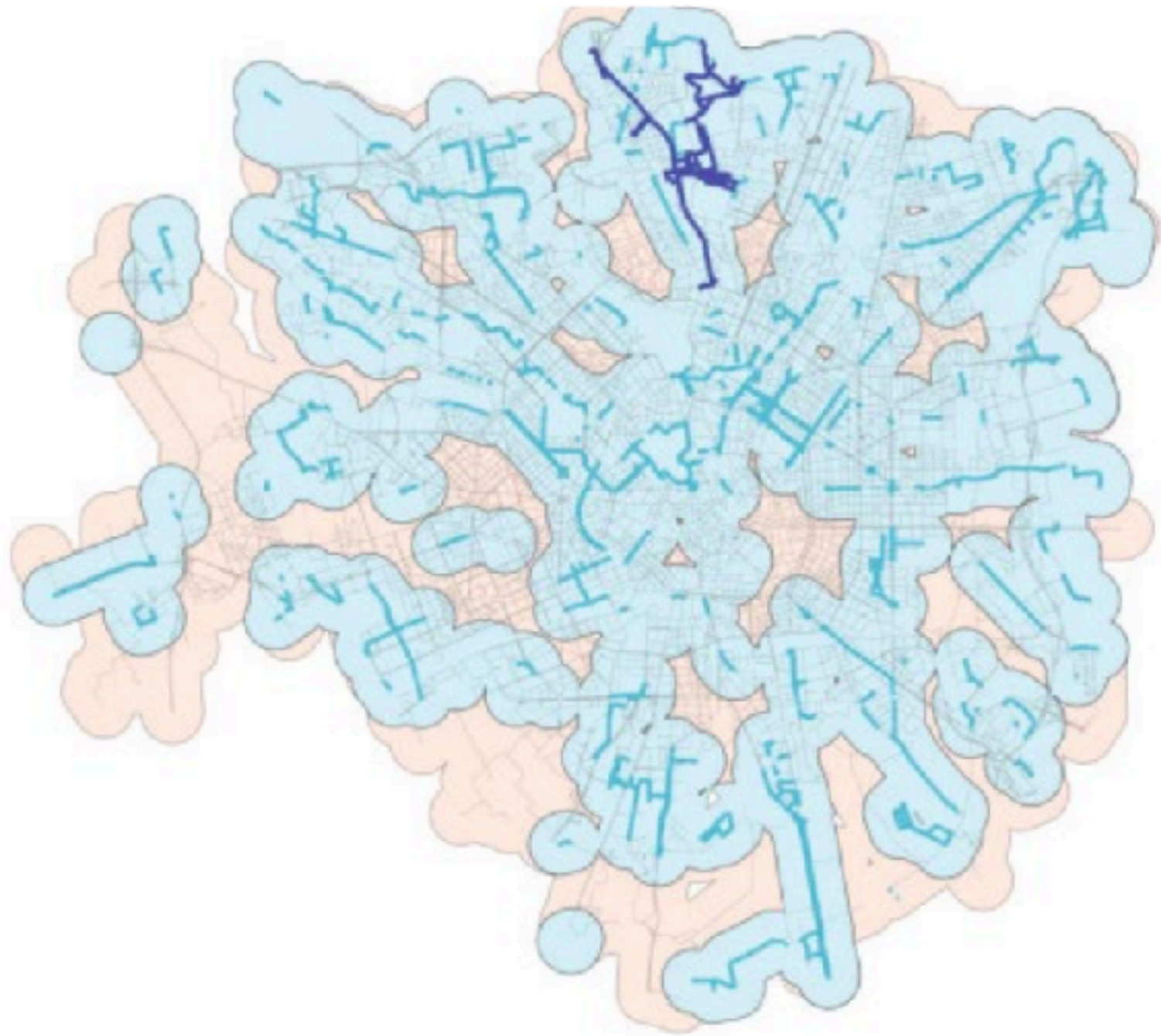
THE POTENTIAL Cycling is cost-efficient and time-tested



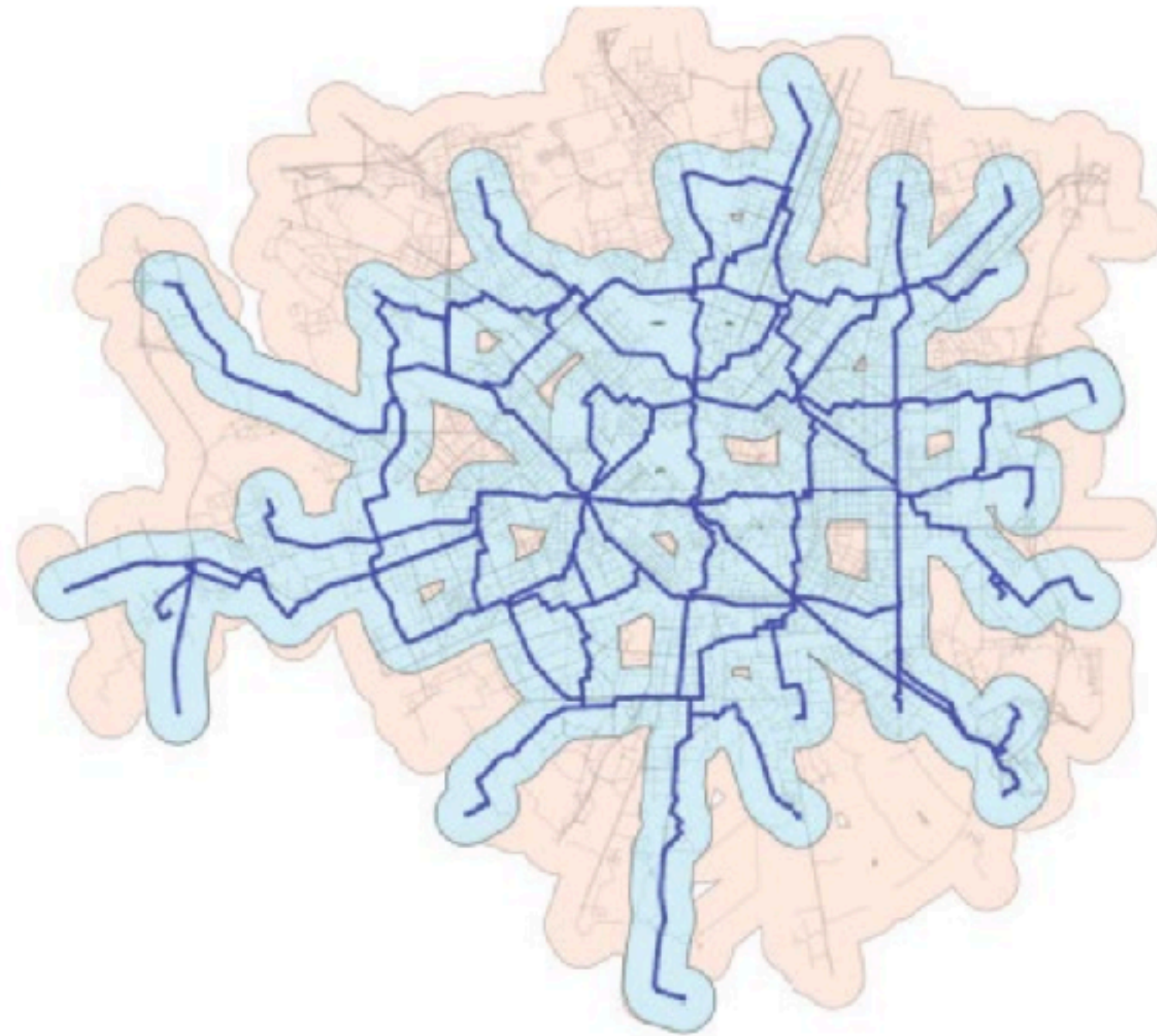
185km
>90 Mio.€

1 km of bike lane:
Breaks even after 3y
Then, **+430000€ / y!**

THE PROBLEM Cities struggle with bike network planning



Real Milan



Synthetic Milan

185km
>90 Mio.€

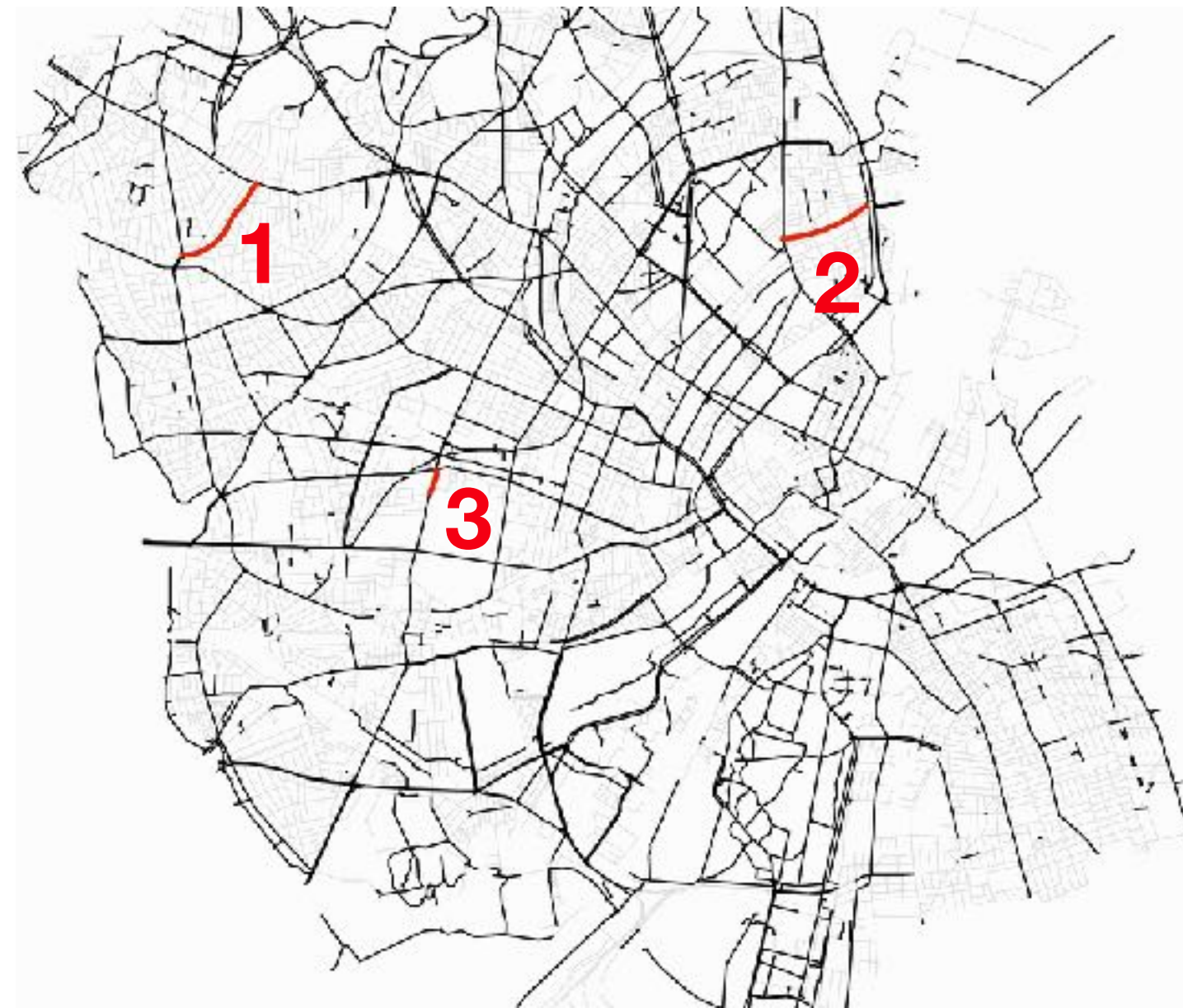
Our research shows:
Data-driven planning is
highly cost-efficient!

THE PROBLEM Cities struggle with bike network planning

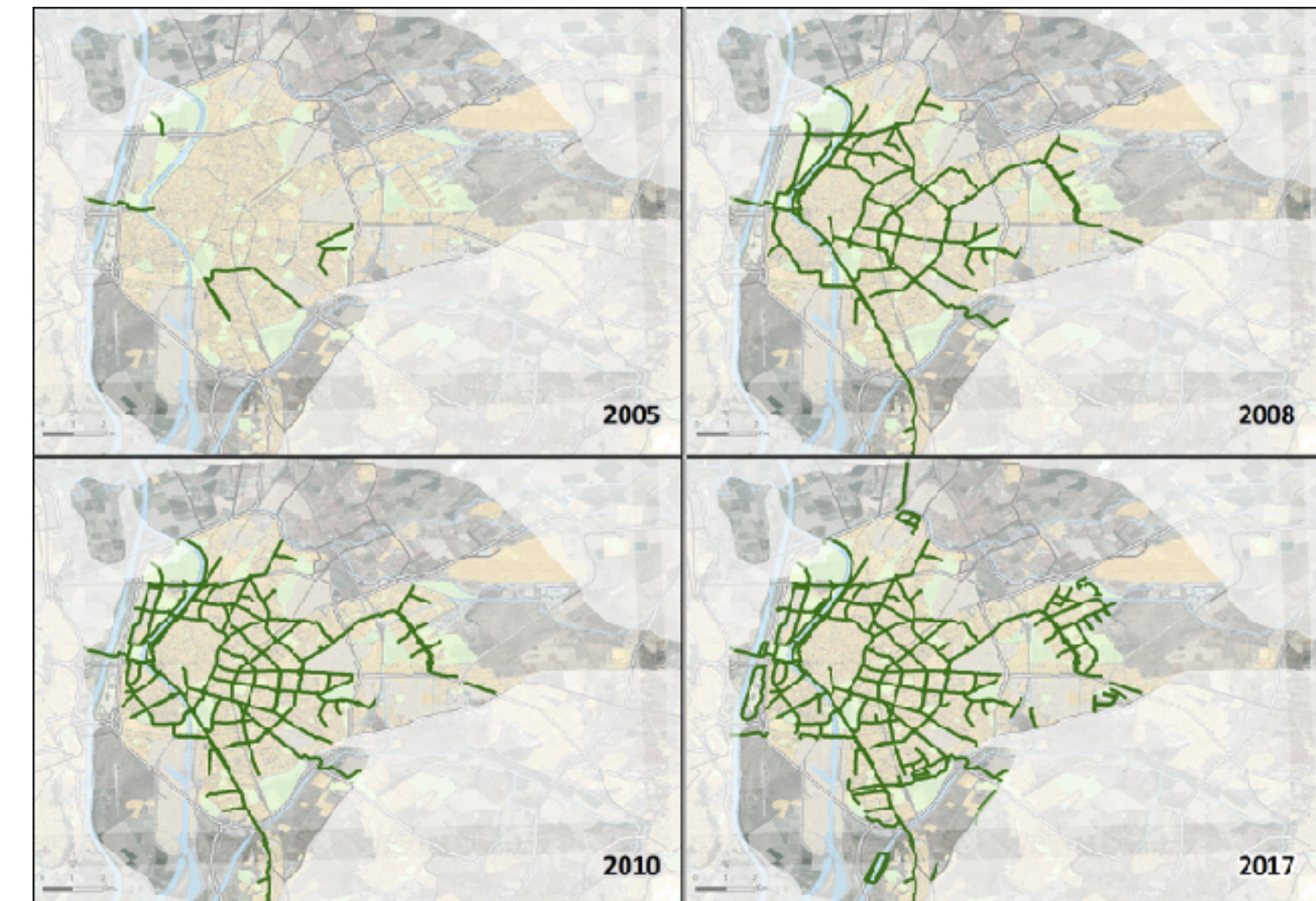
Current network planning is:

- manual
- political
- lacks success metrics

Where to
close the gaps?

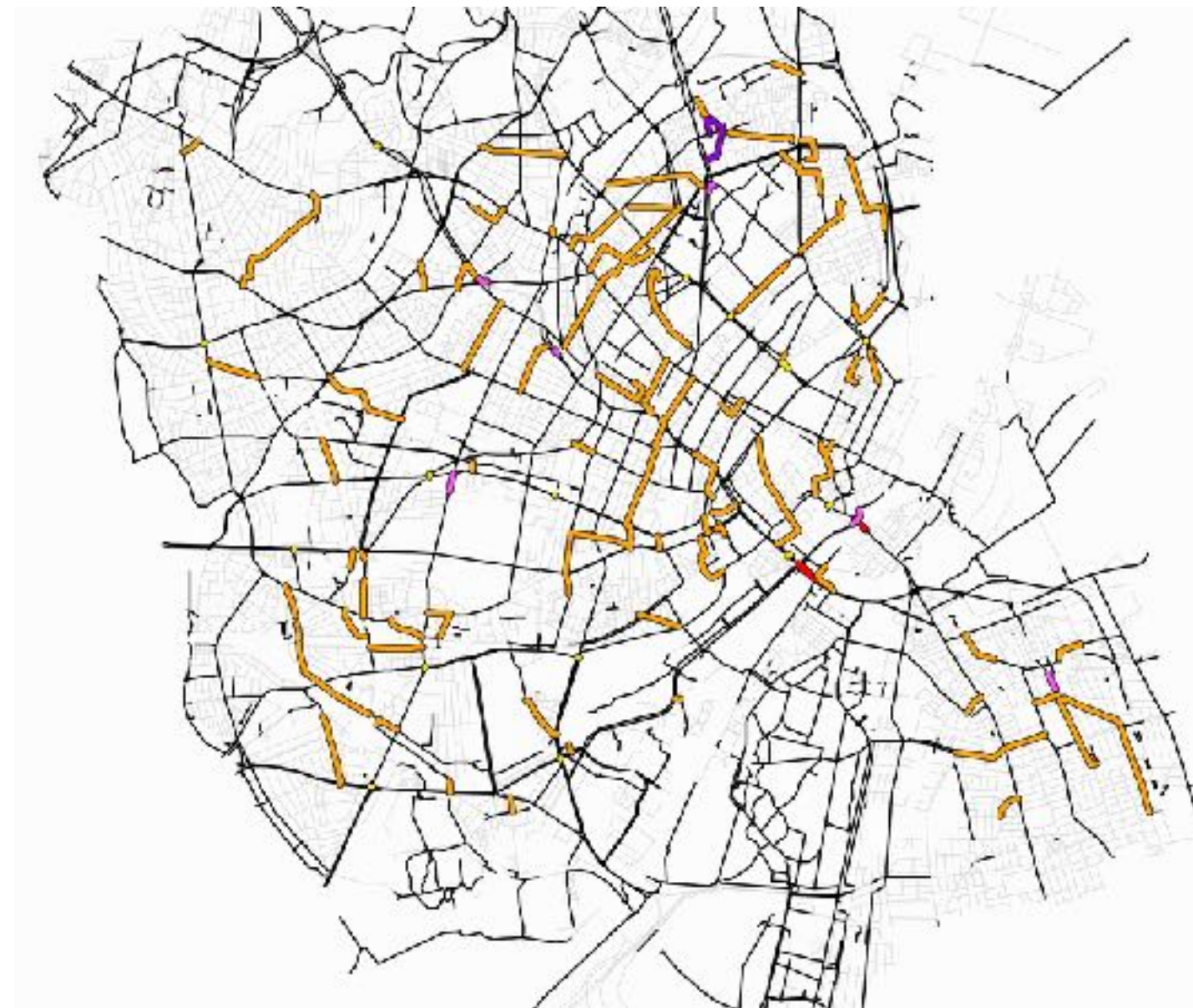


How to
build from scratch?

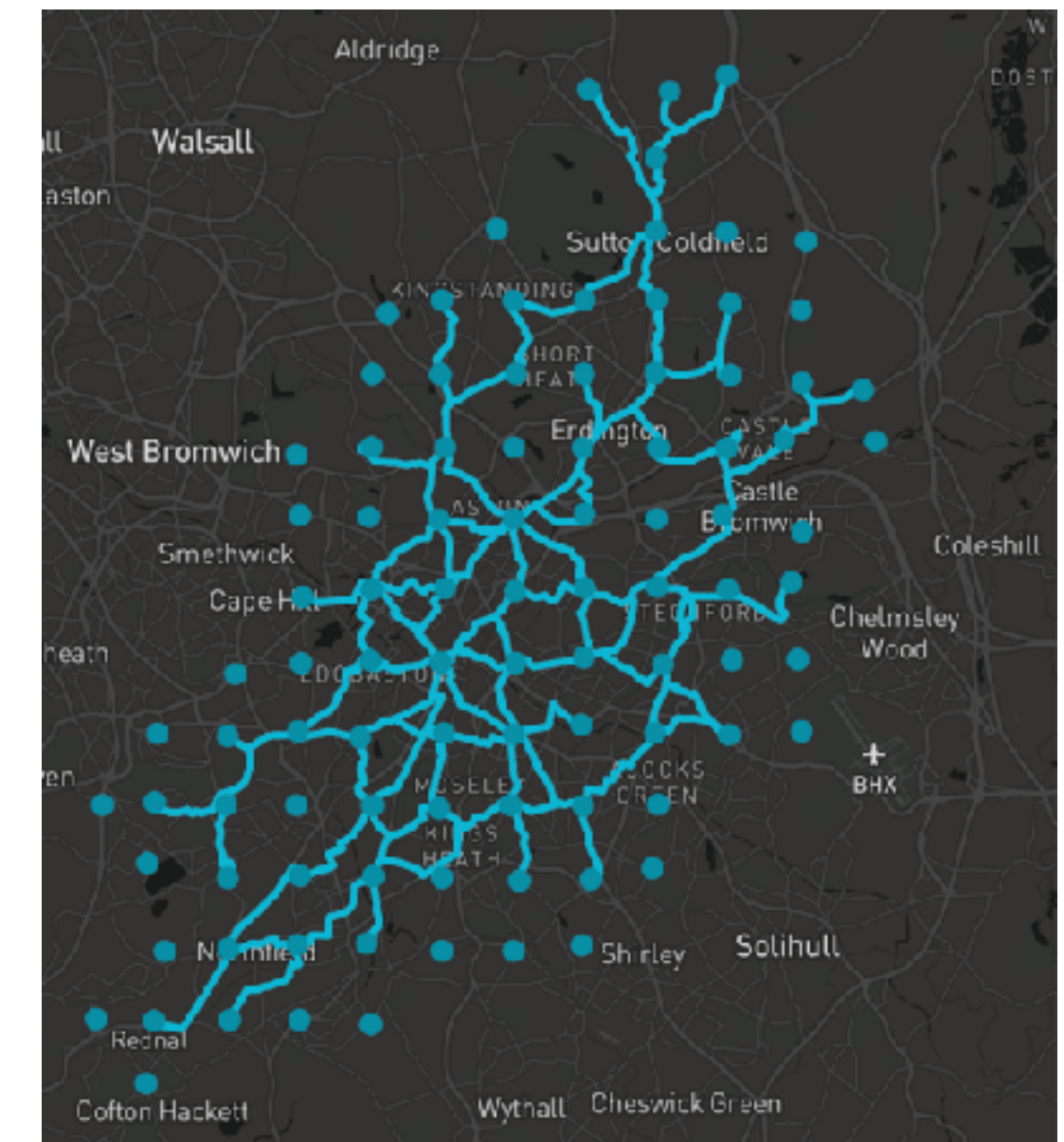


OUR SOLUTION Bicycle network algorithms, supporting planners

Here to
close the gaps!



This way to
build from scratch!



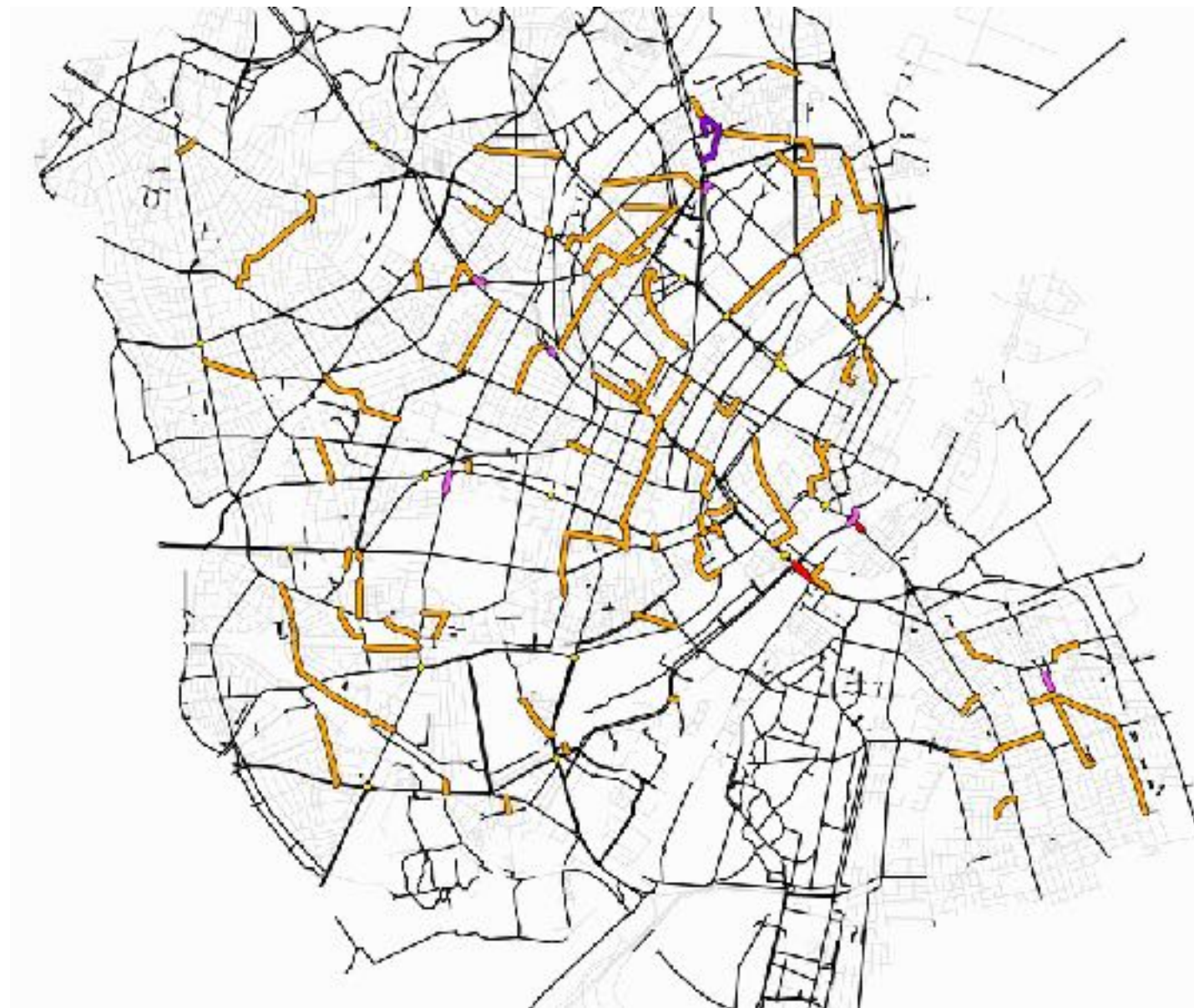
OUR SOLUTION Bicycle network algorithms, supporting planners

We have pioneered a
Science of Bicycle Networks

Key research/algorithms

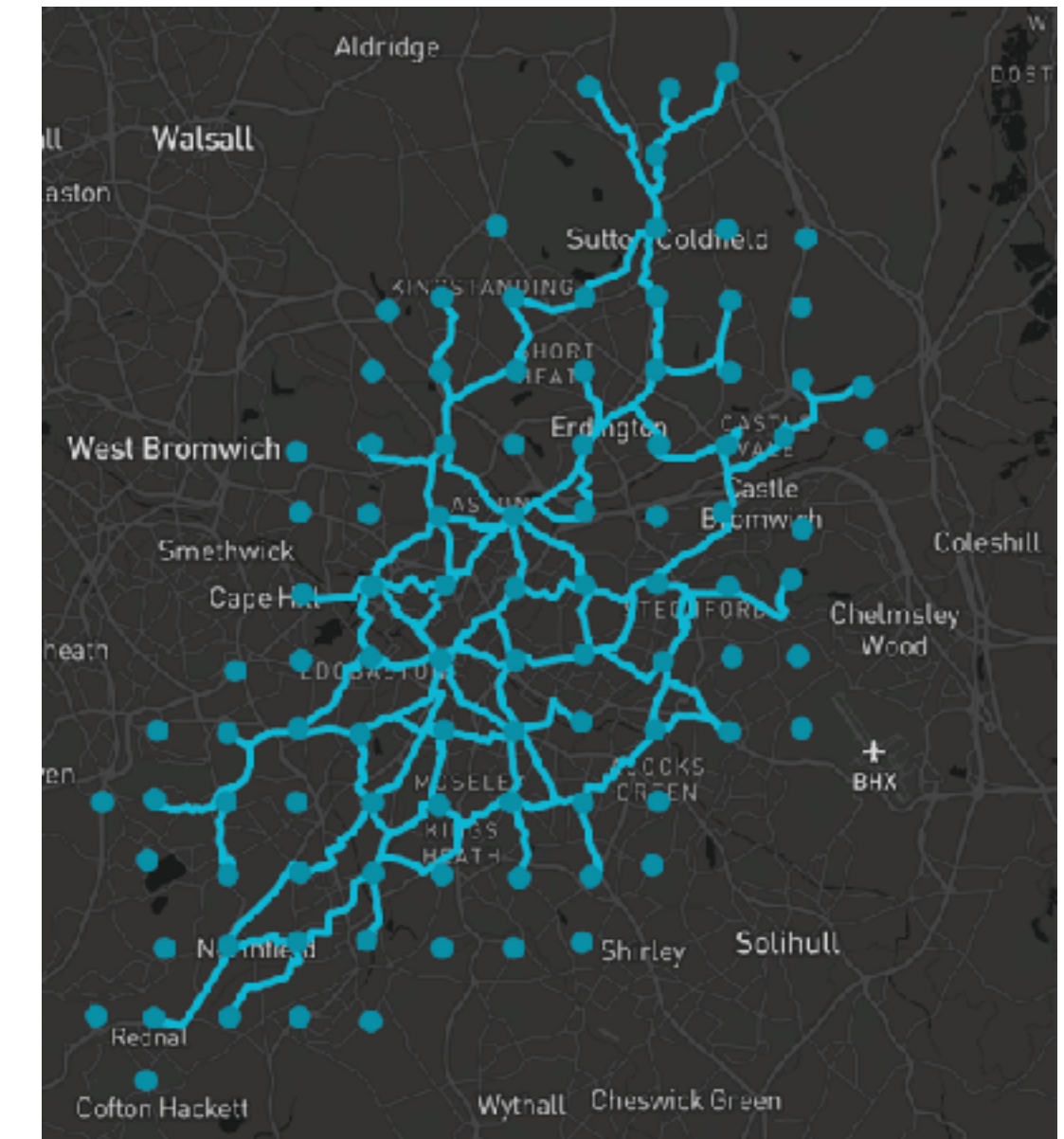
Sebastiao et al, Appl Net Sci (2026)
Szell et al, arxiv:2604.07029 (2026)
Vybornova et al, EPB 52 (2025)
Lonardi et al, R Soc Int 22 (2025)
Vierø et al, EPB 51 (2024)
Büth et al, JOSS 9 (2024)
Folco et al, EPB 50 (2023)
Vybornova et al, Geog Anal 55 (2023)
Szell et al, Sci Rep 12 (2022)
Breum et al, Transp F 56683 (2022)
Natera et al, R Soc Open Sci 7 (2020)

Here to
close the gaps!



FixBike.net

This way to
build from scratch!



GrowBike.net

We visualized 2 algorithms, exciting urban planners!

OUR PROBLEM Our algorithms are not yet user-friendly

- code
 - .ipynb_checkpoints
 - 01_prepare_networks.ipynb
 - 02_prepare_pois.ipynb
 - 03_poi_based_generation.ipynb
 - 04_analyze_results.ipynb
 - 05_plot_results.ipynb
 - 06_video_generation.ipynb
 - X_carconstricted_metrics.ipynb
 - X_export_data.ipynb
 - X_paper_figures_multicity.ipynb
 - X_sandbox.ipynb
 - X_supplements.ipynb
 - functions.py
 - path.py
 - setup.py
 - parameters

Download and wrangle data

Networks

```
In [ ]: for placeid, placeinfo in tqdm(cities.items(), desc = "Cities"):
        if placeinfo["nominatimstring"] != '':
            location = ox.geocoder.geocode_to_gdf(placeinfo["nominatimstring"])
            location = fill_holes(extract_relevant_polygon(placeid, shapely.geometry.shape(location['geometry'][0])))
            if debug: # Draw location polygons and their holes
                try:
                    color = cm.rainbow(np.linspace(0,1,len(location)))
                    for poly,c in zip(location, color):
                        plt.plot(*poly.exterior.xy, c = c)
                        for intr in poly.interiors:
                            plt.plot(*intr.xy, c = "red")
                except:
                    plt.plot(*location.exterior.xy)
                    plt.show()
            else:
                # https://gis.stackexchange.com/questions/113799/how-to-read-a-shapefile-in-python
                shp = fiona.open(PATH["data"] + placeid + "/" + placeid + ".shp")
                first = next(iter(shp))
                try:
                    location = Polygon(shapely.geometry.shape(first['geometry'])) # If shape file is given as linestring
                except:
                    location = shapely.geometry.shape(first['geometry'])

Gs = {}
for parameterid, parameterinfo in tqdm(osmnxparameters.items(), desc = "Networks", leave = False):
    for i in range(0,10): # retry
        try:
```


OUR PROBLEM Our algorithms are not yet user-friendly

code

- .ipynb_checkpoints
 - 01_prepare_networks.ipynb
 - 02_prepare_pois.ipynb
 - 03_poi_based_generation.ipynb
 - 04_analyze_results.ipynb
 - 05_plot_results.ipynb
 - 06_video_generation.ipynb
 - X_carconstricted_metrics.ipynb
 - X_export_data.ipynb
 - X_paper_figures_multicity.ipynb
 - X_sandbox.ipynb
 - X_supplements.ipynb
- functions.py
- path.py
- setup.py
- parameters

Download and wrangle data

Networks

```
In [ ]: for placeid, placeinfo in tqdm(cities.items(), desc = "Cities"):
        if placeinfo["nominatimstring"] != '':
            location = ox.geocoder.geocode_to_gdf(placeinfo["nominatimstring"])
            location = fill_holes(extract_relevant_polygon(placeid, shapely.geometry.shape(location['geometry'][0])))
            if debug: # Draw location polygons and their holes
                try:
                    color = cm.rainbow(np.linspace(0,1,len(location)))
                    for poly,c in zip(location, color):
                        plt.plot(*poly.exterior.xy, c = c)
                        for intr in poly.interiors:
                            plt.plot(*intr.xy, c = "red")
                except:
                    plt.plot(*location.exterior.xy)
                    plt.show()
            else:
                # https://gis.stackexchange.com/questions/113799/how-to-read-a-shapefile-in-python
                shp = fiona.open(PATH["data"] + placeid + "/" + placeid + ".shp")
                first = next(iter(shp))
                try:
                    location = Polygon(shapely.geometry.shape(first['geometry'])) # If shape file is given as linestring
                except:
                    location = shapely.geometry.shape(first['geometry'])

Gs = {}
for parameterid, parameterinfo in tqdm(osmnxparameters.items(), desc = "Networks", leave = False):
    for i in range(0,10): # retry
        try:
```

Jupyter notebooks



Spaghetti code



OUR PROBLEM Our algorithms are not yet user-friendly

code

- .ipynb_checkpoints
 - 01_prepare_networks.ipynb
 - 02_prepare_pois.ipynb
 - 03_poi_based_generation.ipynb
 - 04_analyze_results.ipynb
 - 05_plot_results.ipynb
 - 06_video_generation.ipynb
 - X_carconstricted_metrics.ipynb
 - X_export_data.ipynb
 - X_paper_figures_multicity.ipynb
 - X_sandbox.ipynb
 - X_supplements.ipynb
- functions.py
- path.py
- setup.py
- parameters

Download and wrangle data

Networks

```
In [ ]: for placeid, placeinfo in tqdm(cities.items(), desc = "Cities"):
        if placeinfo["nominatimstring"] != '':
            location = ox.geocoder.geocode_to_gdf(placeinfo["nominatimstring"])
            location = fill_holes(extract_relevant_polygon(placeid, shapely.geometry.shape(location['geometry'])[0]))
            if debug: # Draw location polygons and their holes
                try:
                    color = cm.rainbow(np.linspace(0,1,len(location)))
                    for poly,c in zip(location, color):
                        plt.plot(*poly.exterior.xy, c = c)
                        for intr in poly.interiors:
                            plt.plot(*intr.xy, c = "red")
                except:
                    plt.plot(*location.exterior.xy)
                    plt.show()
            else:
                # https://gis.stackexchange.com/questions/113799/how-to-read-a-shapefile-in-python
                shp = fiona.open(PATH["data"] + placeid + "/" + placeid + ".shp")
                first = next(iter(shp))
                try:
                    location = Polygon(shapely.geometry.shape(first['geometry'])) # If shape file is given
                except:
                    location = shapely.geometry.shape(first['geometry'])

Gs = {}
for parameterid, parameterinfo in tqdm(osmnxparameters.items(), desc = "Networks", leave = False):
    for i in range(0,10): # retry
        try:
```

Jupyter notebooks

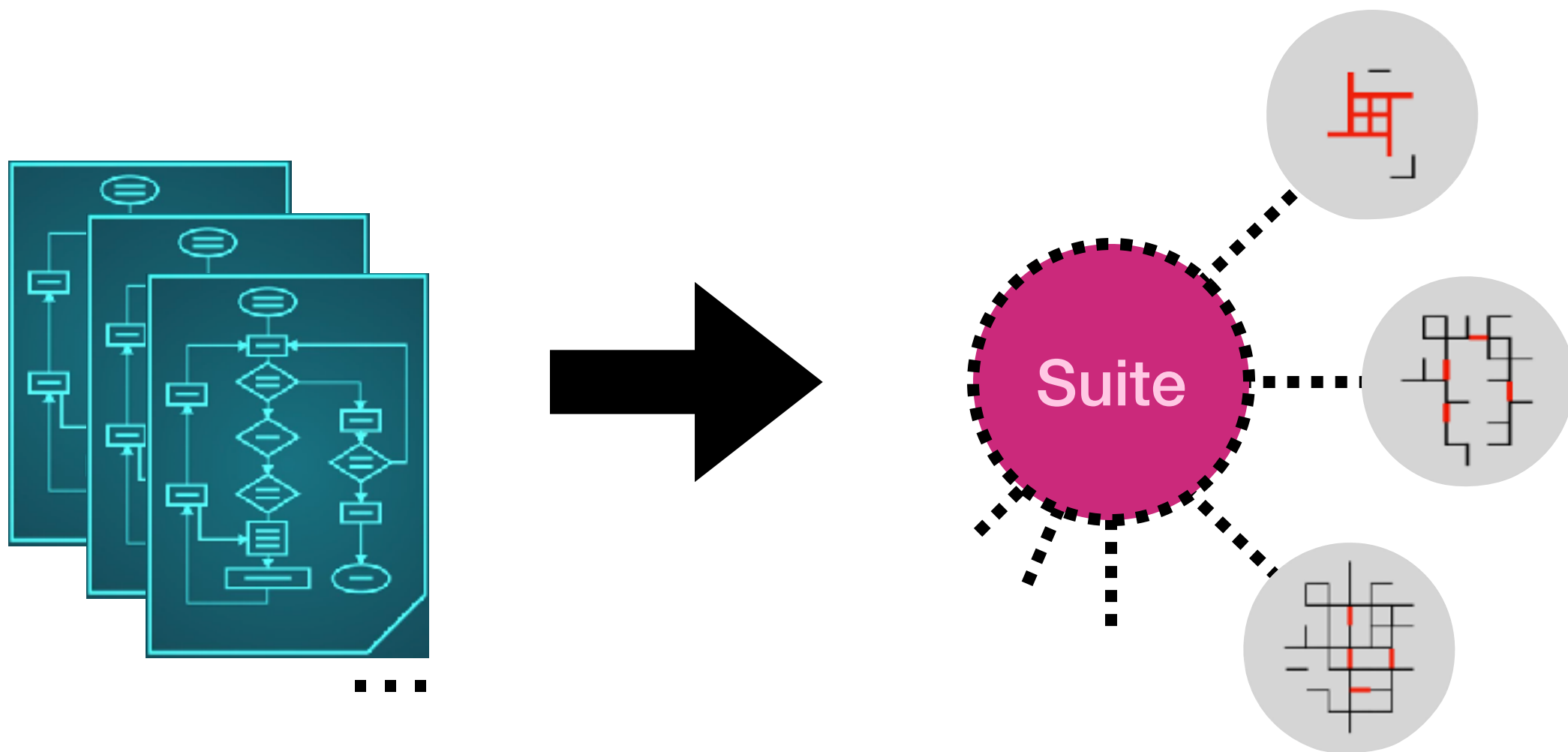


Spaghetti code



We got funds to 1) Create usable software

Innovation Fund Denmark

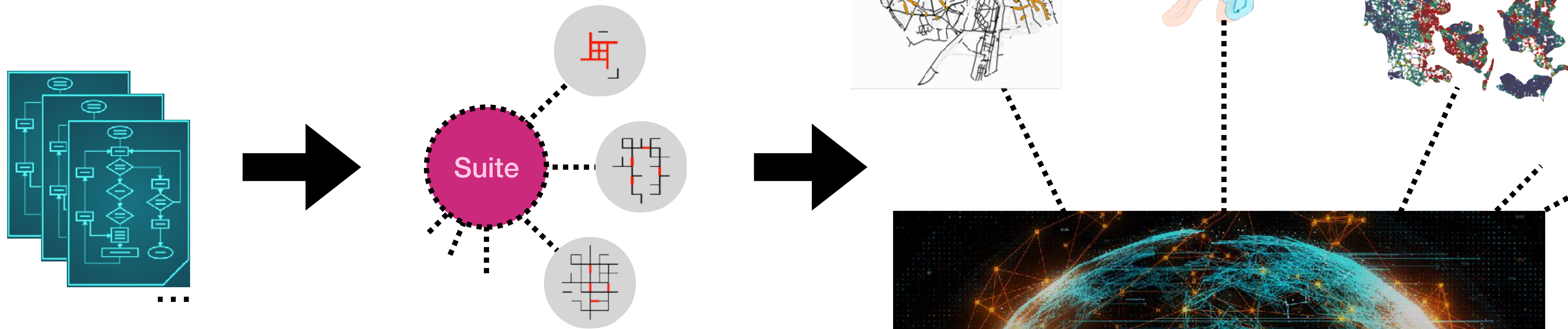


1) Research software engineer (Manuel)
creates packaged Python software

Usable by planners

We got funds to 1) Create usable software, 2) Visualize on the web

nnovation Fund Denmark



1) Research software engineer (Manuel) creates packaged Python software

Usable by planners

2) Web developer (Moritz) visualizes our solutions on a web platform

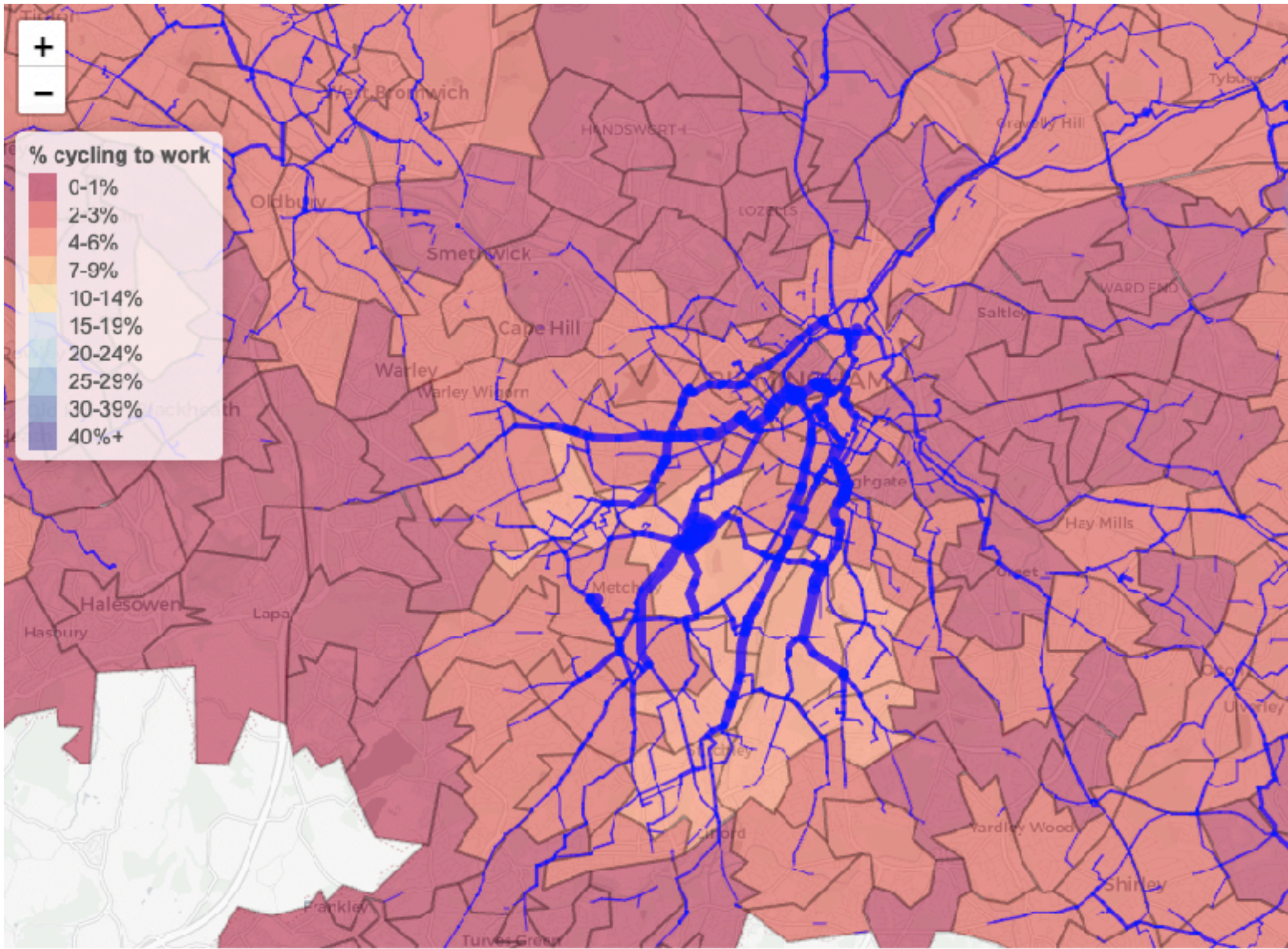
Guides planners, inspires citizens

We are not the first: Few fantastic free tools exist!

A/B Street

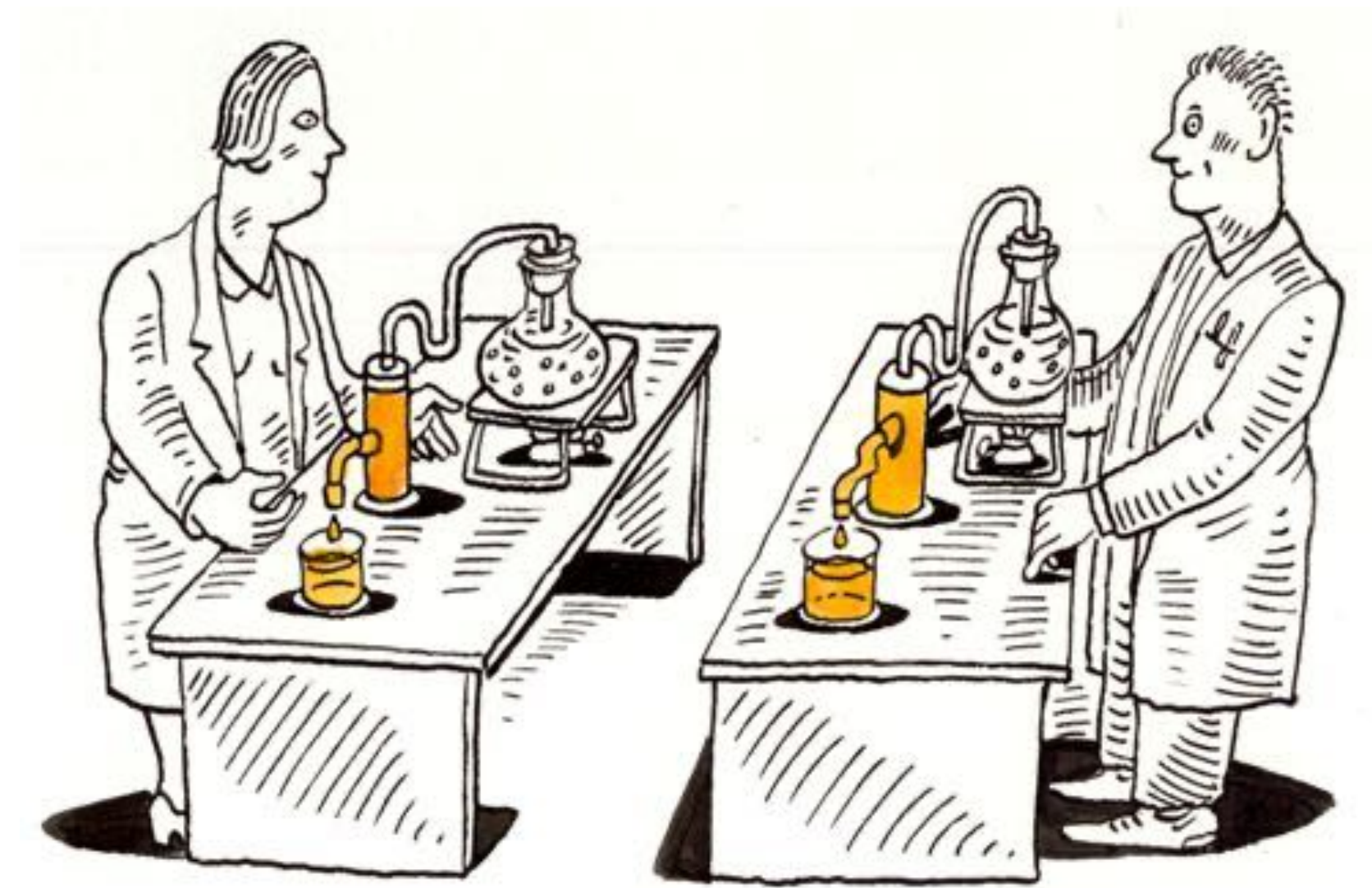


Propensity to Cycle Tool



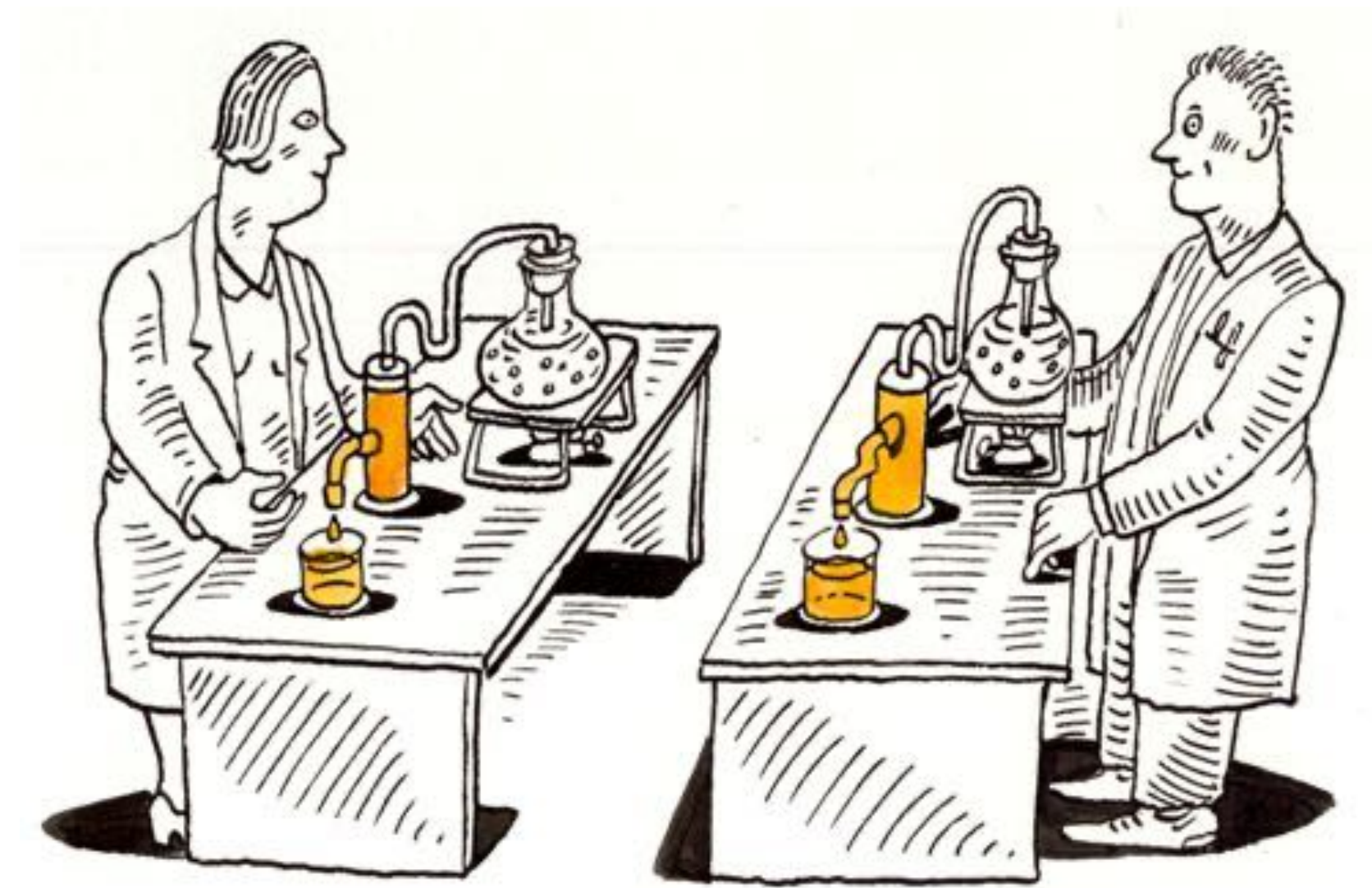
Our philosophy: Free, Open-Source, Reproducible

The software should be usable, modifiable, and reproducible by **anyone**, including proactive citizens, to create a compelling vision for urban cycling in their city.



Our philosophy: Free, Open-Source, Reproducible

The software should be usable, modifiable, and reproducible by **anyone**, including proactive citizens, to create a compelling vision for urban cycling in their city.



It should be an **alternative to extractive practices** where cities overpay consultants for a one-off report



Our tools

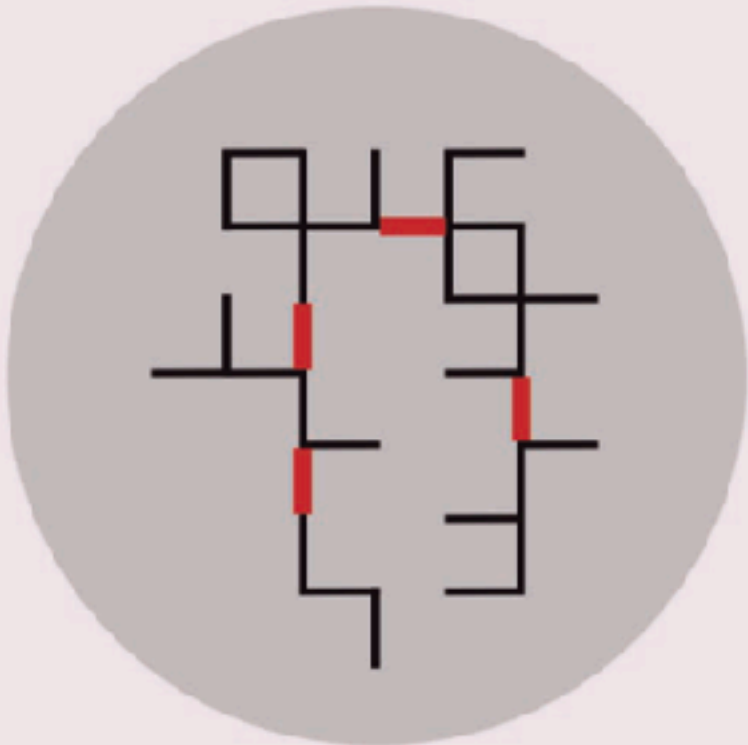
In 2026 we are turning 6+ algorithms into Python packages

bikenetkit.org

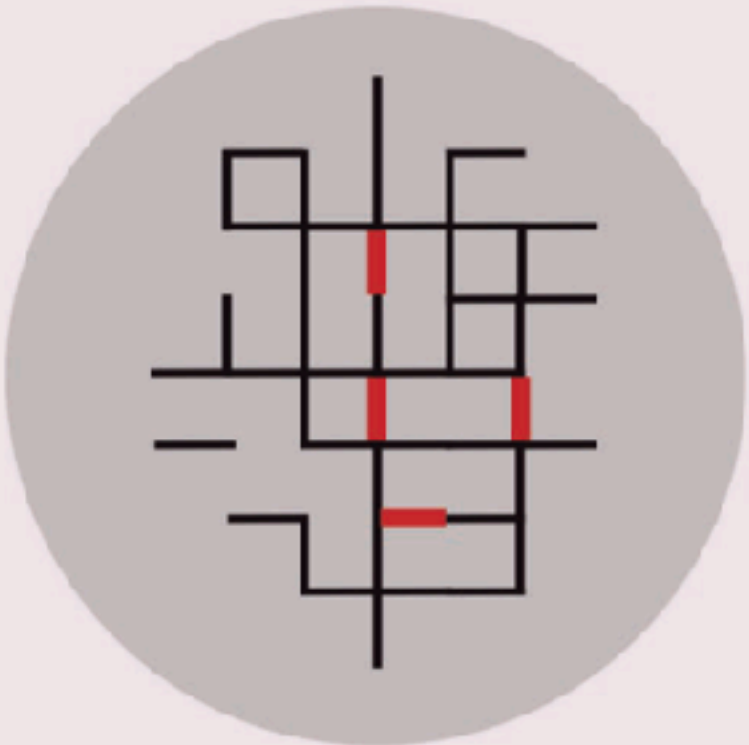
GrowBikeNet



LinkBikeNet



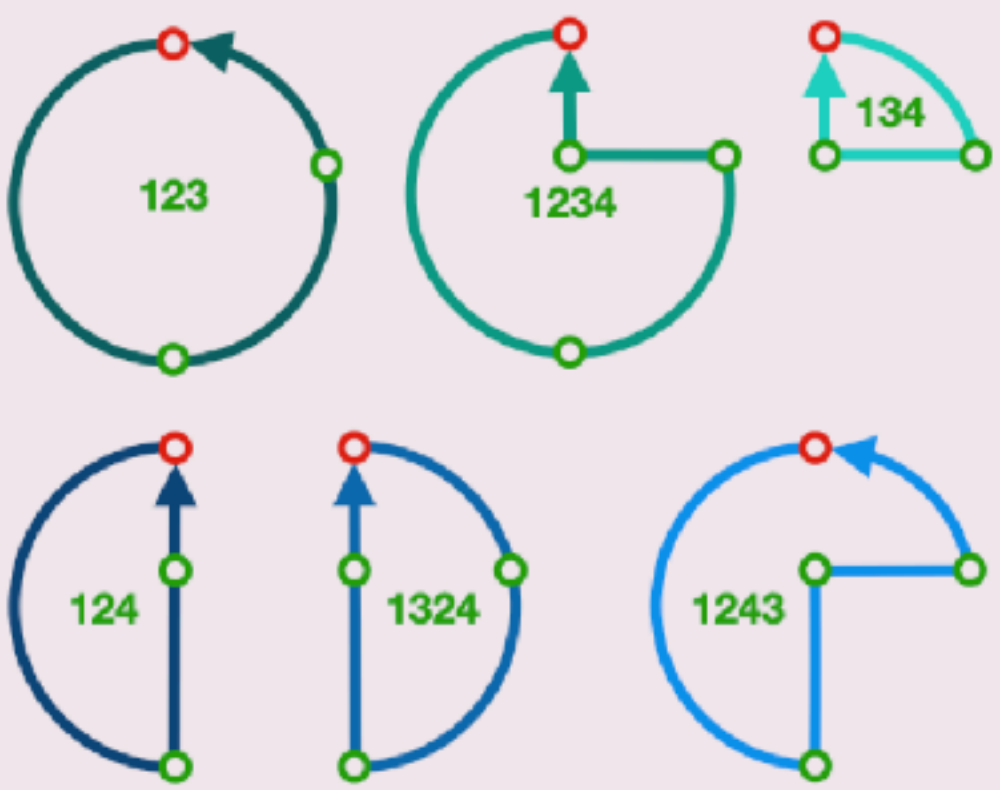
FixBikeNet



GrowBikeNetPlus



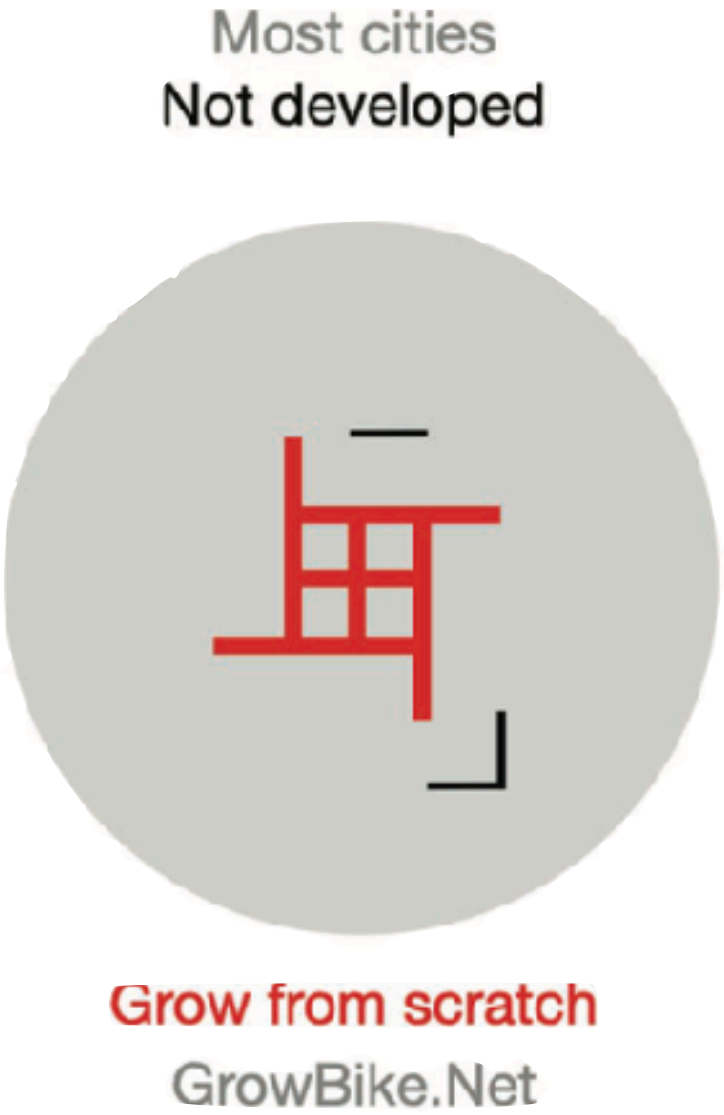
LoopBikeNet



+ more in the future...

Different cities need different network design approaches

GrowBikeNet



Different cities need different network design approaches

GrowBikeNet

Birmingham

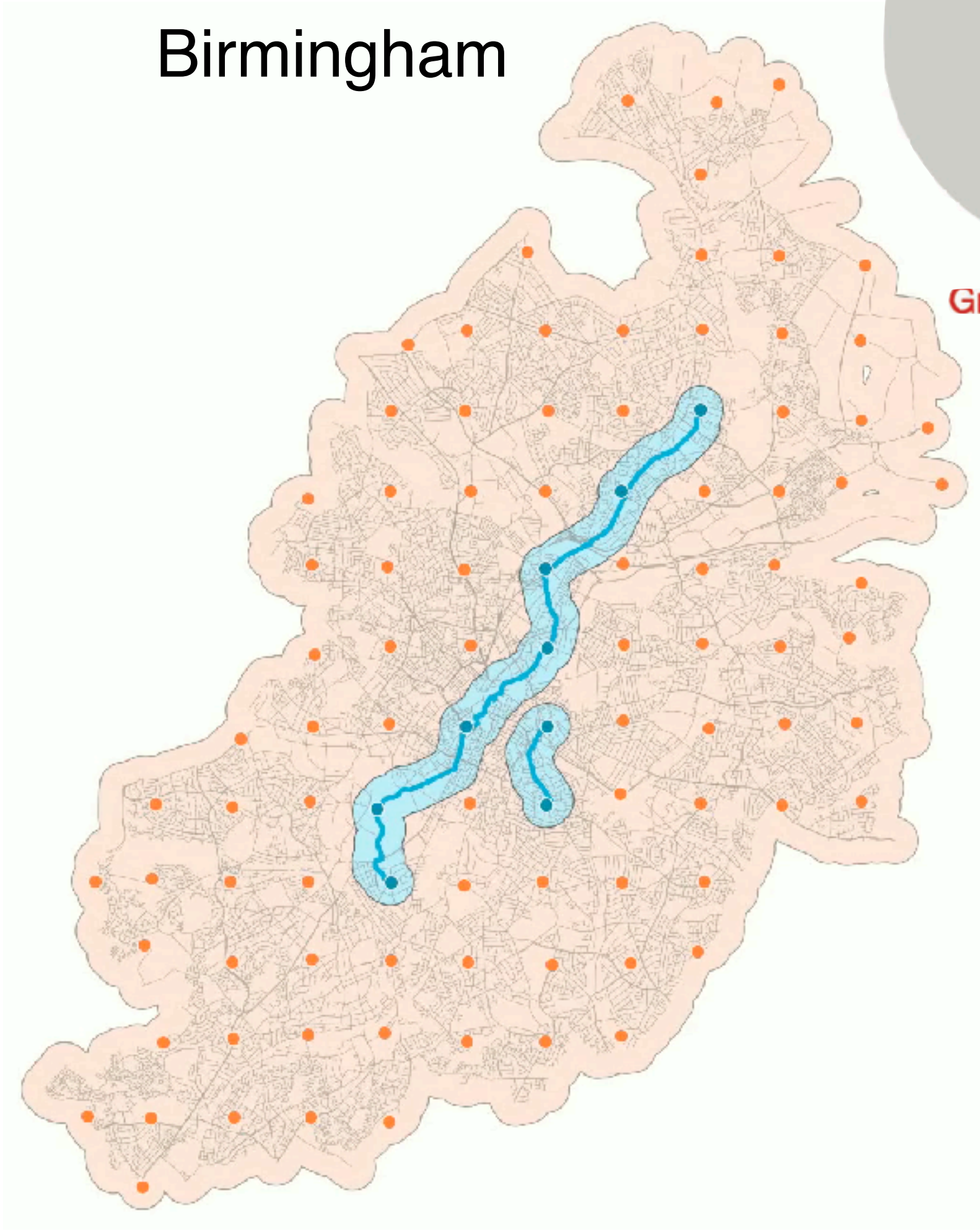
Most cities
Not developed



Grow from scratch
GrowBike.Net

Different cities need different network design approaches

Birmingham

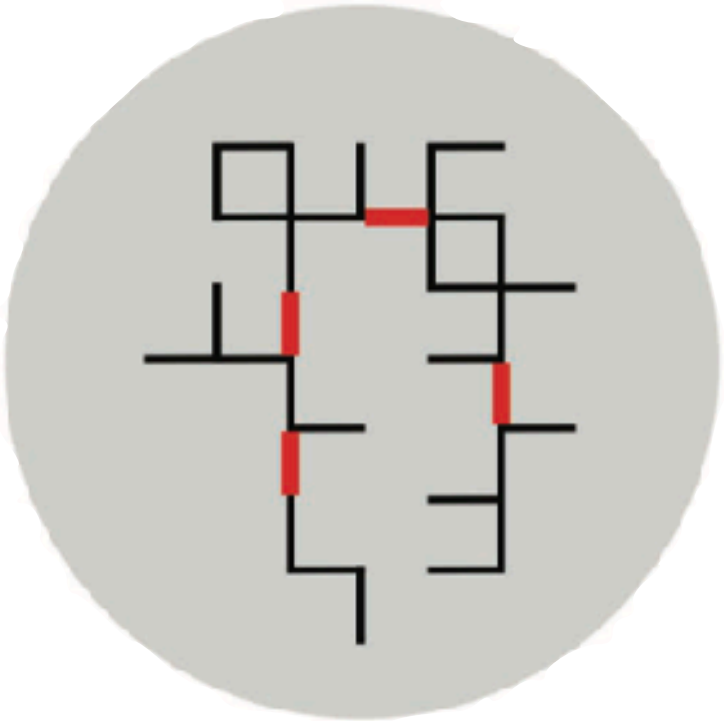


Most cities
Not developed



Grow from scratch
GrowBike.Net

Some cities
Developed but
disconnected



Connect components
LinkBike.Net

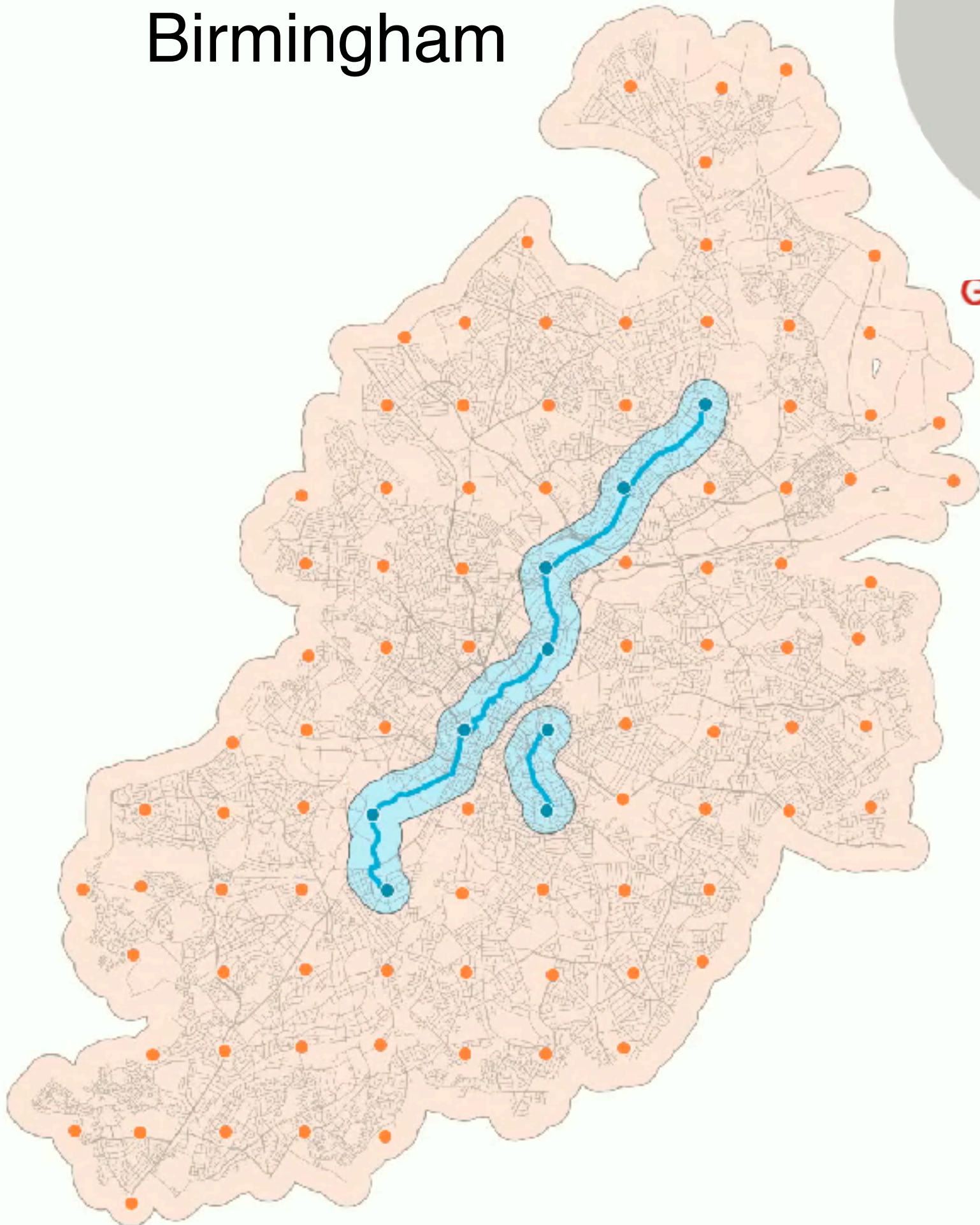
Budapest



Different cities need different network design approaches

FixBikeNet

Birmingham

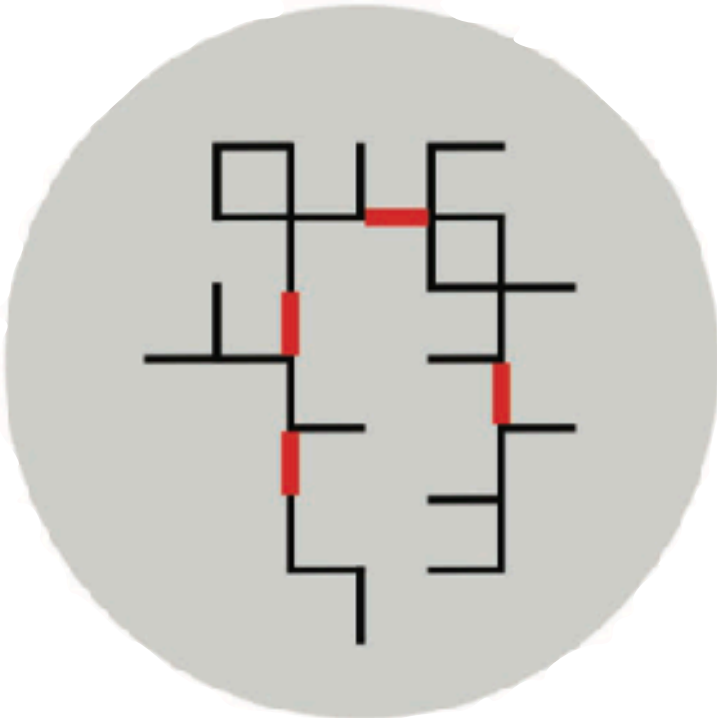


Most cities
Not developed



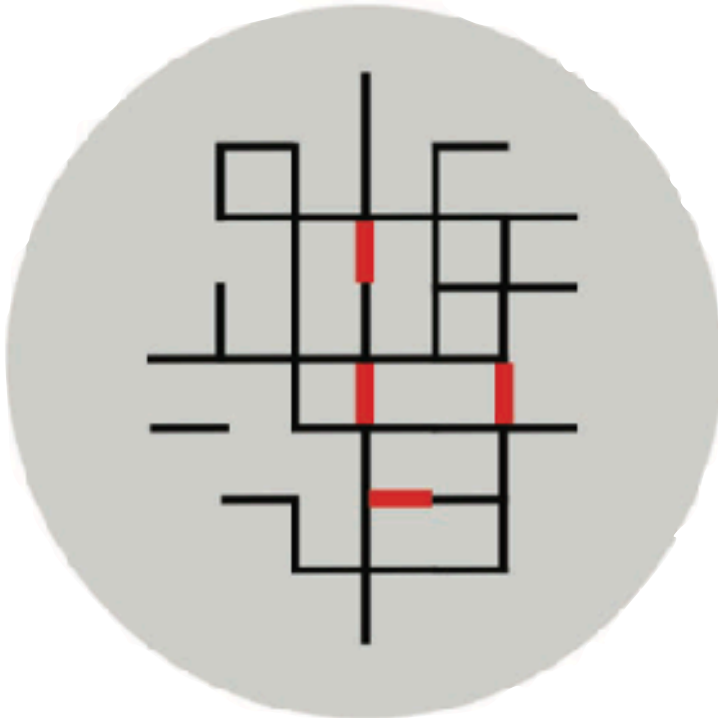
Grow from scratch
GrowBike.Net

Some cities
Developed but
disconnected



Connect components
LinkBike.Net

Few cities
Developed and
mostly connected



Find missing links
FixBike.Net

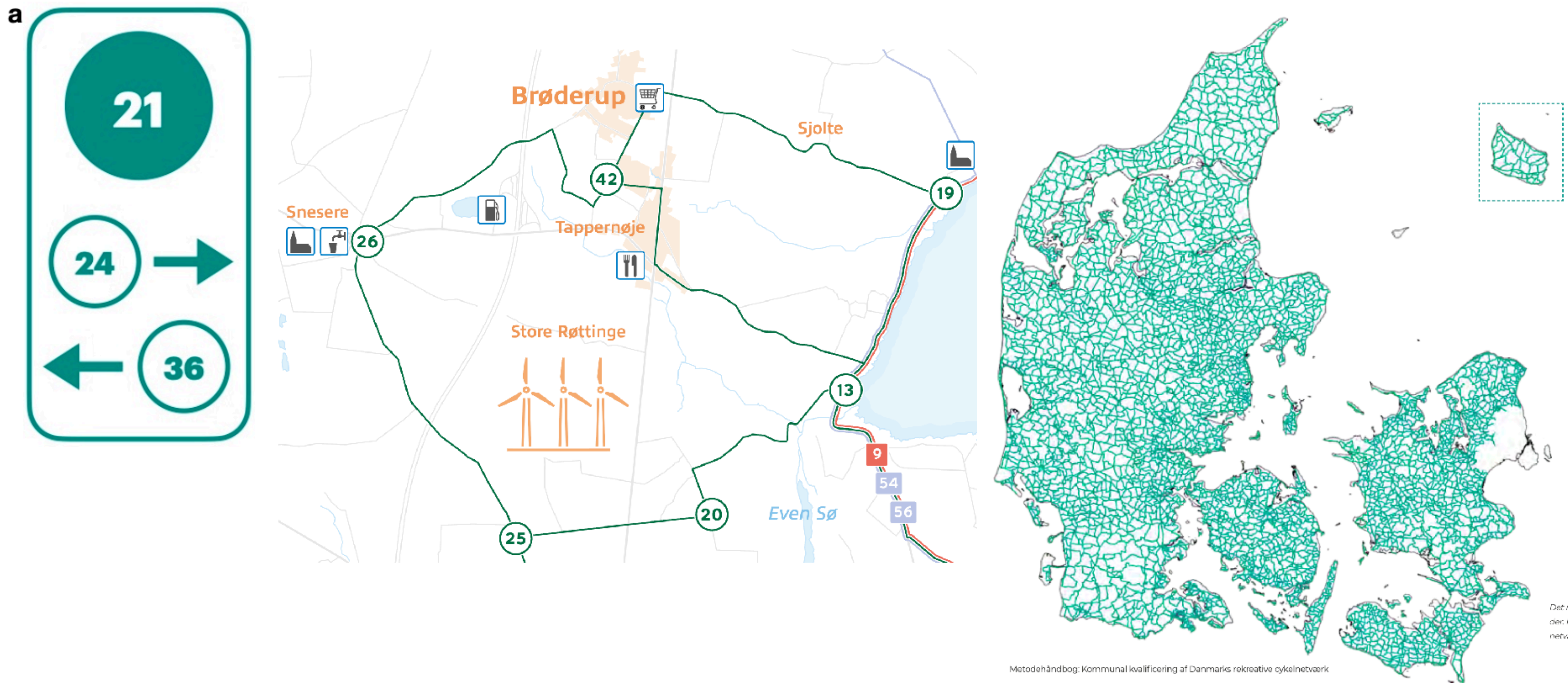
Budapest



Copenhagen

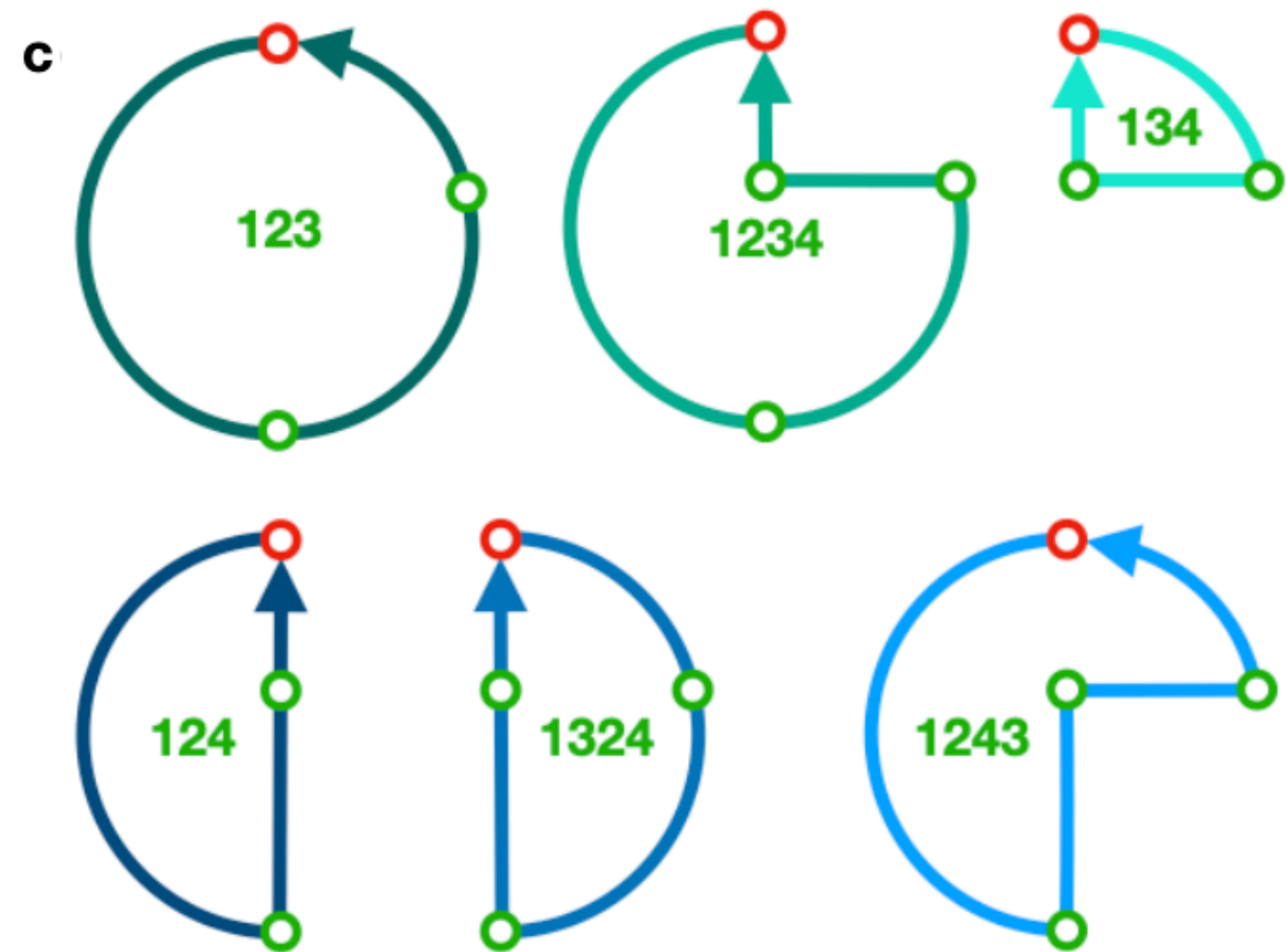
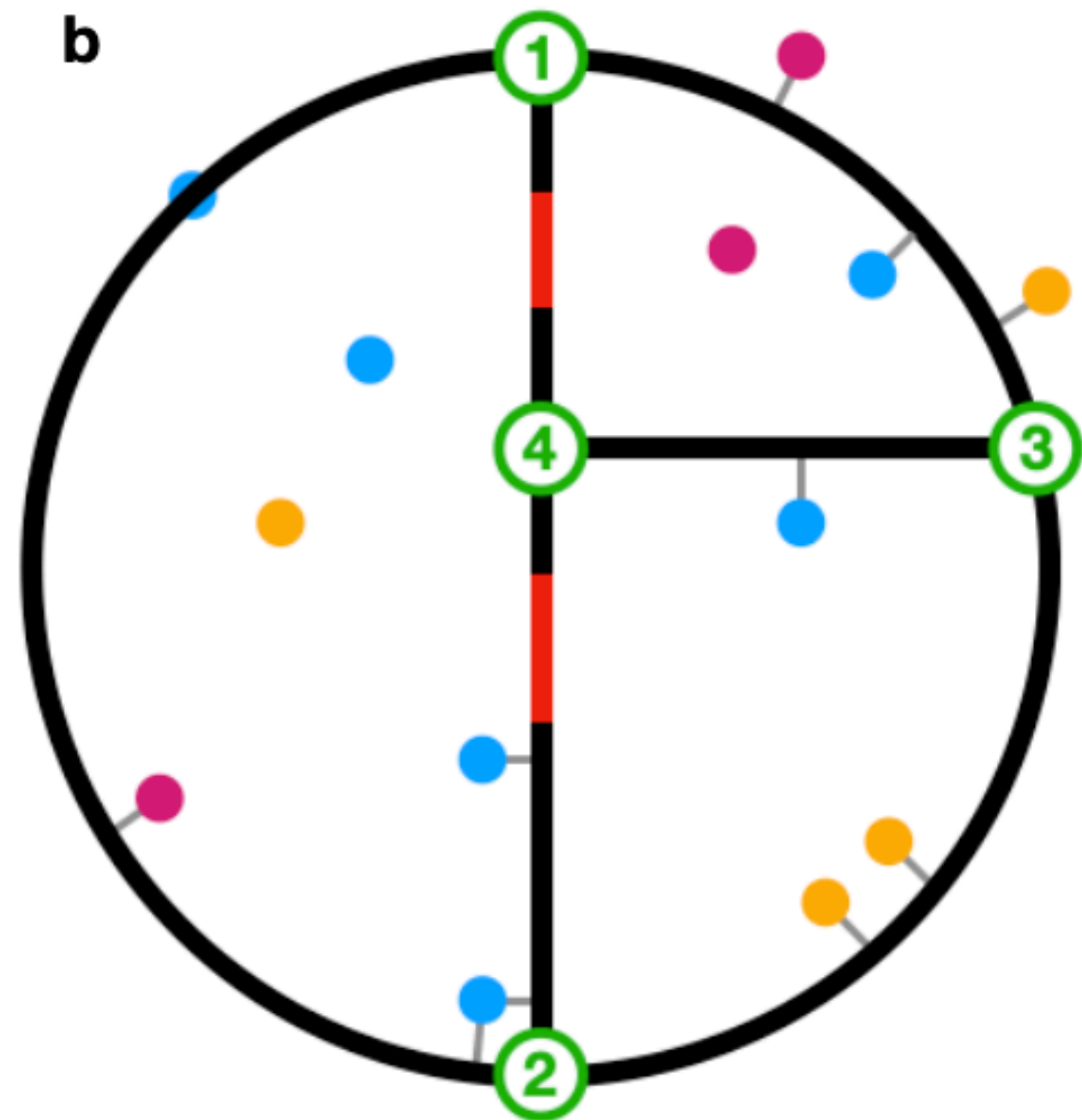
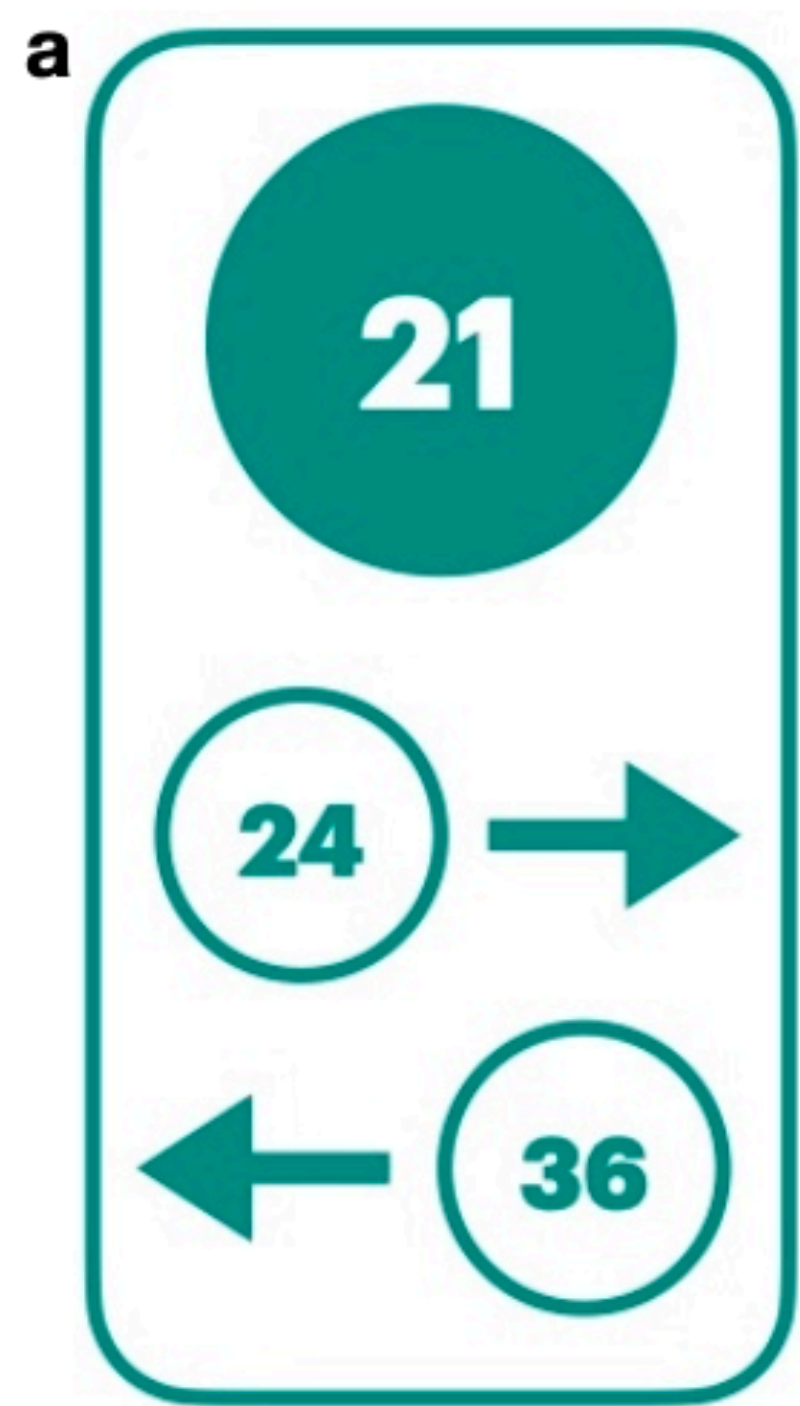


Regional, recreational networks are a different beast



Regional, recreational networks are a different beast

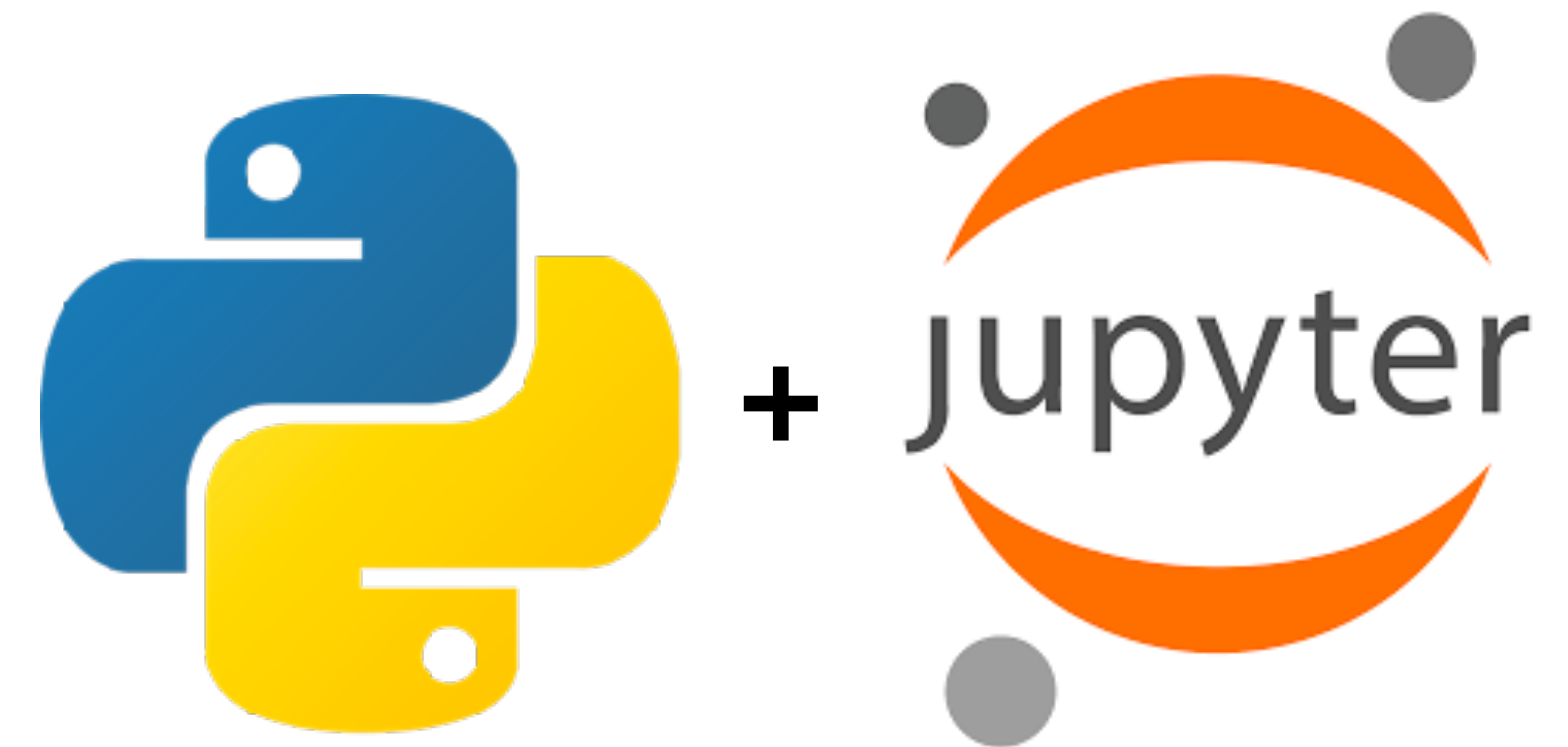
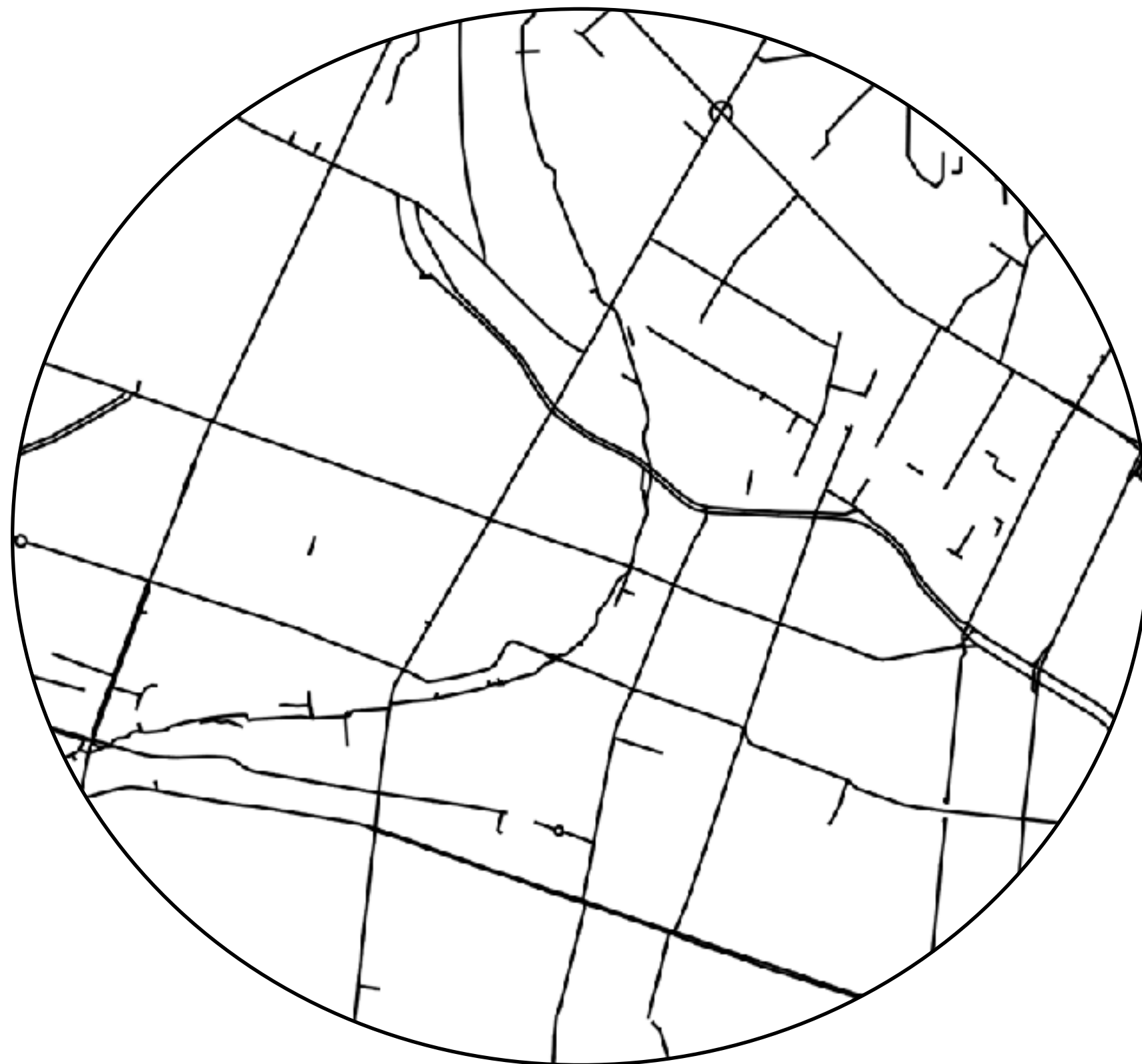
LoopBikeNet



Loop census: All roundtrips from one node

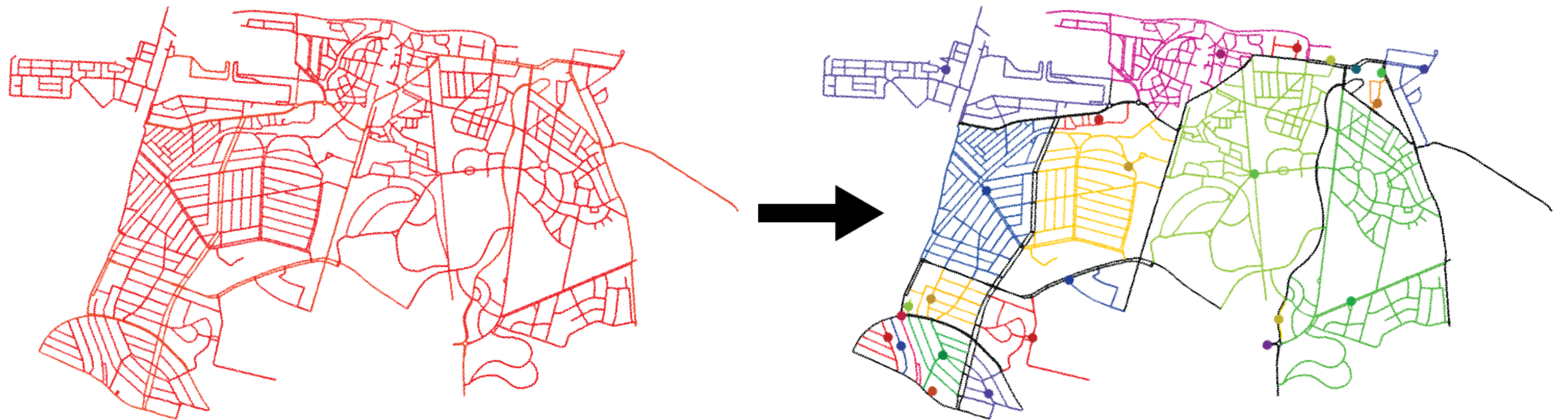


Bicycle Infrastructure Data & Network Assessment

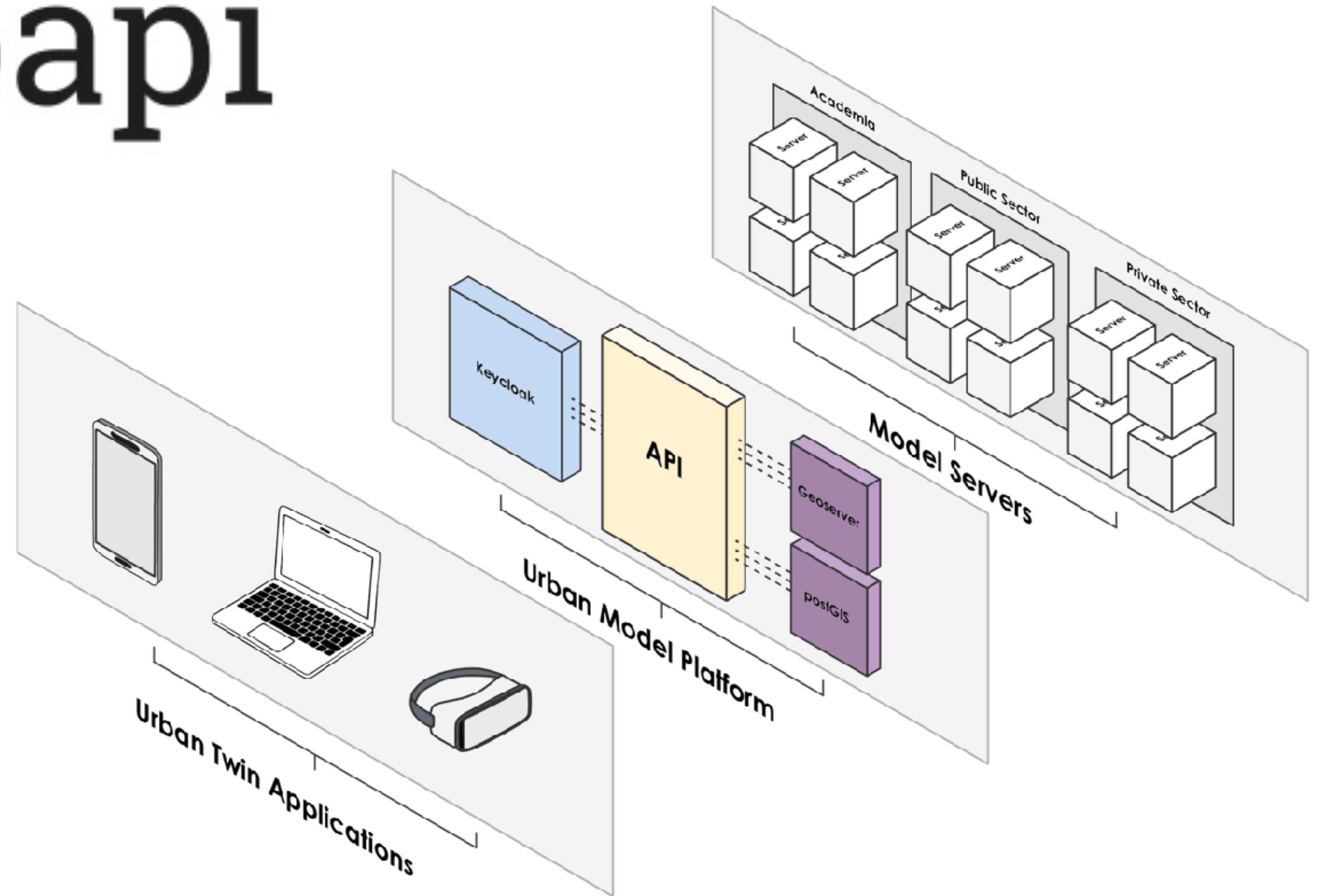


Funded by The Danish Road Directorate 

SUPERBLOCIFY



Apart from packaged code + viz platform, we plan an API





Networks, Data, and Society

Research group at IT University of Copenhagen



We are researching network and data science applications to social systems

We are interdisciplinary: Data Science, Physics, Mathematics, GIS,...

We are alumni of MIT, Harvard, Cornell, Northeastern, Bell Labs, UN

We won the Young Academy's prize Research Environment of the Year 2022

VILLUM FONDEN

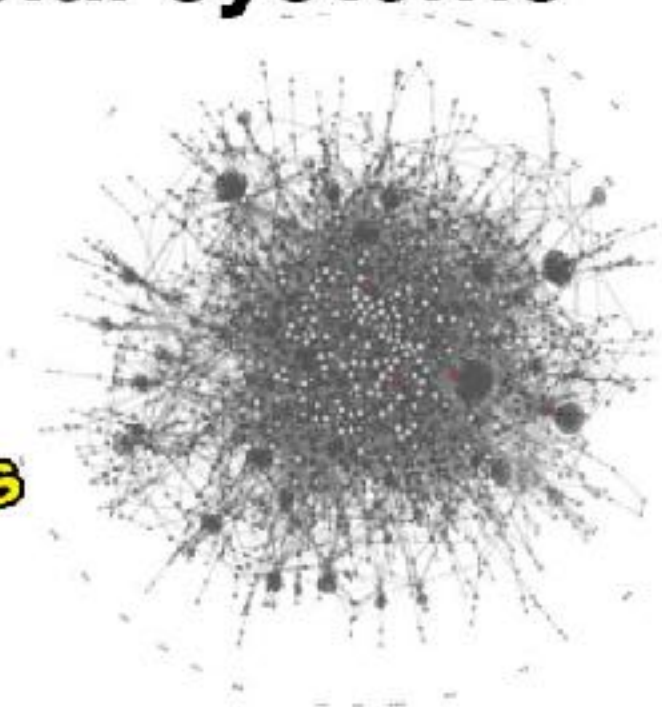
CARLSBERG FOUNDATION



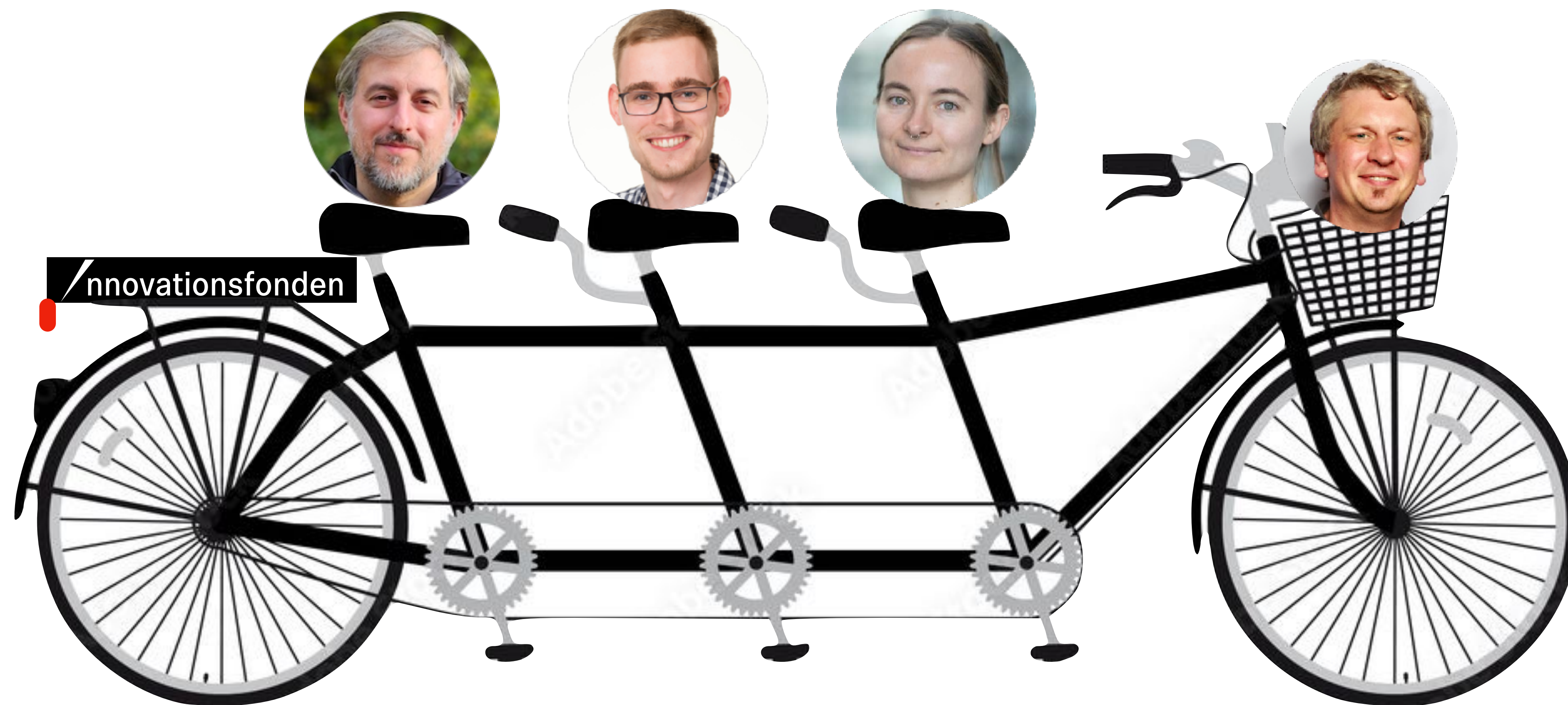
/nnovationsfonden



Social Networks
Bicycle Networks
Complex Systems
NLP



Help us spread the word, help us develop the code!



bikenetkit.org

Give us critical feedback, help us test the code!



michael@szell.net